

Nexis: A Multi-Agent Autonomous Engineering Platform with Closed-Loop Fault Recovery

by

Ratul Sikder

Submitted to the Institute of Information Technology (IIT)

in partial fulfillment of the degree requirements

at the

UNIVERSITY OF DHAKA

August 2026

© University of Dhaka 2026. All rights reserved.

The author hereby grants the Institute of Information Technology (IIT), University of Dhaka, permission to reproduce and distribute copies of this project report, in whole or in part, for academic purposes.

Nexis: A Multi-Agent Autonomous Engineering Platform with Closed-Loop Fault Recovery

by

Ratul Sikder

Submitted to the Institute of Information Technology (IIT) on 11 August 2026,
in partial fulfillment of the degree requirements

This is to declare that this project is the author's original work. No part of it has been submitted elsewhere, in whole or in part, for the award of any other degree or diploma, and the plagiarism policy stated by the supervisor has been maintained.

Certified by

Project Supervisor

Abstract

Modern software engineering teams spend an estimated 40 to 60 percent of their time on maintenance, debugging, and manual incident response, rather than on building new capability. A number of AI-assisted developer tools already exist, such as code completion assistants, automated test generators, and deployment dashboards, but each of these addresses only an individual task in isolation. None of them model the collaborative structure of a real engineering team, and none of them close the loop between detecting a production failure and actually repairing it. **Nexis** has been designed to address both of these gaps together. The system models a software engineering organisation as nine role-specialised autonomous agents, arranged across two cooperating layers: a five-agent Execution Team (Architect, Backend, Quality Assurance (QA), DevOps, and Data Engineer) and a four-agent Self-Healing Loop (Sentinel, Pathfinder, Synthesiser, and Validator). These agents are durably orchestrated using Temporal, and every automated repair is gated by a severity-routed, human-in-the-loop Approval Gate that supports Approve, Reject, and Modify (Reinforcement Learning from Human Feedback (RLHF)) decisions. This report documents the complete, working system, which includes a multi-tenant control plane built with Go (Golang), PostgreSQL with Row-Level Security (RLS), Neo4j, and Redis; a Docker shadow-validation sandbox; a real GitHub-App-mediated GitOps deployment path; and a from-scratch Docker preview-deploy engine. It also includes a Next.js console that spans authentication, Role-Based Access Control (RBAC), incident observability, billing, and multi-provider integrations. Two live end-to-end recovery runs against a local Large Language Model (LLM) have been reported verbatim, alongside a live and verified proof of the preview-deploy engine. What has been deliberately left out of scope, namely a fully credentialed production deployment to the registered `nexis.sh` domain and the human-subject NASA Task Load Index (NASA-TLX) study, has been stated plainly rather than left implied.

Keywords: Multi-agent systems, autonomous software engineering, self-healing systems, closed-loop fault recovery, root-cause analysis, LLM-guided program repair, DevOps automation, Temporal workflow orchestration, human-in-the-loop AI.

Table of Contents

Chapter 1: Introduction.....	5
1.1 Background.....	5
1.2 Motivation.....	5
1.3 Problem Statement.....	6
1.4 Project Objectives.....	6
1.5 Scope of the Project.....	6
1.6 Significance of the Project.....	7
Chapter 2: Literature Review.....	8
2.1 Overview.....	8
2.2 Research Gap.....	9
2.3 Conclusion.....	10
Chapter 3: System Analysis and Requirements.....	11
3.1 Requirement Analysis.....	11
3.2 Functional Requirements.....	11
3.3 Non-Functional Requirements.....	13
3.4 System Feasibility.....	14
Chapter 4: System Design and Architecture.....	16
4.1 System Context.....	16
4.2 Component Architecture.....	16
4.3 Data and Storage Architecture.....	17
4.4 Self-Healing Loop Pipeline.....	18
4.5 Deployment and Infrastructure.....	19
4.6 Use Case Diagrams.....	20
4.7 Data Flow Diagrams.....	21
4.8 Self-Healing Loop Sequence Diagram.....	24
Chapter 5: Implementation.....	25
5.1 Tools and Technology.....	25
5.2 Project View.....	25
5.3 Database Schema.....	40
Chapter 6: Testing and Results.....	41
6.1 Static Verification.....	41
6.2 Live End-to-End Recovery Runs.....	41
6.3 Live Deploy-Engine Verification.....	42
6.4 Defects Found and Fixed During This Engineering Cycle.....	43
6.5 Testing Summary.....	43
Chapter 7: Conclusion and Future Work.....	44
7.1 Overview.....	44
7.2 Limitations of the Project.....	44
7.3 Future Work.....	45
7.4 Conclusion.....	45
References.....	46

Chapter 1: Introduction

This chapter lays out why **Nexis** was built in the first place: the pain points that drove it, where existing tools fall short, the objectives set for the project, and who, human and non-human alike, it actually serves. It ends by explaining why a genuinely closed loop, running from fault detection through to verified deployment, actually matters.

1.1 Background

Software rarely fails at the moment it is written. It fails later, in production, under load, against data and dependencies that its authors never anticipated. It may be noted that this cost is not merely anecdotal: the original proposal for this project draws on industry experience placing it at 40–60% of an engineering team’s time, spent on maintenance, debugging, and incident response rather than on building anything new. Consequently, for a small team this means that for every engineer creating value, there is roughly one more engineer employed full-time just to keep it working.

Artificial Intelligence (AI)-assisted software engineering has grown rapidly over the last few years. Code-completion assistants such as GitHub Copilot [1] now write a meaningful share of new code, and the tools built around them generate tests, review pull requests, and summarise logs. Yet each of these remains a point solution: it accelerates one task performed by one human, and nothing more. The production incident lifecycle makes the point clearly. It runs through seven stages: detect a fault, diagnose its root cause, write a fix, validate it, obtain approval, deploy it, and verify the deployment. At every hand-off between these stages, a human engineer remains the transport mechanism; thus, the chain stays manual from end to end.

Nexis exists to close that chain. Rather than assisting one engineer at one task, it models an entire software engineering organisation as nine role-specialised autonomous agents, arranged in two cooperating layers: a five-agent Execution Team and a four-agent Self-Healing Loop, both durably orchestrated by the Temporal workflow engine [2]. The human approval decision, in turn, is reduced to a single severity-routed click rather than an open-ended manual process.

1.2 Motivation

The pain points behind this project are not abstract; they show up in some form in almost any engineering organisation:

- **No tool closes the loop.** Monitoring products such as Sentry [14], Datadog [16], and PagerDuty [15] can detect a failure and page a human about it; code assistants such as GitHub Copilot [1] write code, but only once a human has asked for it. Nothing on the market currently connects the two, so that a detected failure leads on its own to an autonomous repair.
- **The engineer is the bottleneck for every incident.** Even a routine alert requires a human to read the logs, localise the fault, write a patch, run the tests, request review, deploy, and then watch the rollout. That human sits on the critical path for all seven stages, even for fault classes that recur in an almost identical form each time.
- **Root-cause analysis is unaided.** Localising a fault, in practice, means tracing call chains and service dependencies by hand and from memory, since no mainstream incident tool walks the code dependency graph or ranks candidate root causes against any stated evidence.
- **Repair without validation is dangerous, so it is not attempted.** Because there is no safe, sandboxed path for a machine-generated patch to prove itself before a human ever sees it, organisations reasonably refuse to let machines patch anything at all; as such, the loop stays open.
- **Approval is all-or-nothing.** Deployment gates are typically either absent (which is risky automation) or heavyweight (a full review cycle for what may be a one-line fix), and no existing tool scales the depth of human involvement to the measured severity of the change.

1.3 Problem Statement

Existing approaches to AI-assisted engineering and incident response, taken together, share a set of structural limitations. They are **isolated tools**, in that detection, diagnosis, code generation, testing, and deployment each live in separate products with no shared state or protocol connecting them; they provide **no closed loop**, since a detected fault never automatically becomes a validated, deployable repair; and they keep a **human as the bottleneck for every decision**, however small, repetitive, or low-risk the change may be. They also offer **no causal root-cause tooling**, as nothing traverses the actual dependency structure of the code the way a graph database such as Neo4j [3] makes possible, and nothing applies causal reasoning of the kind formalised by libraries such as DoWhy [4] to explain *why* a fault occurred. Finally, they provide **no safe autonomous deploy path**: even where a machine can propose a patch, there is no sandboxed stage to validate it, no severity-aware approval gate, and nothing to verify or roll back the deployment automatically, so shipping that patch can never really be defended.

Hence the problem this project addresses may be stated as follows: *how can the full production-fault lifecycle (detect, diagnose, repair, validate, approve, deploy, and verify) be closed autonomously by a team of cooperating role-specialised agents, while still keeping a human decision in the loop exactly where, and only where, the severity of the change warrants it?*

1.4 Project Objectives

Nexis set out to build the following, and delivered each as described below:

1. **A nine-agent, two-layer architecture.** A five-agent Execution Team (Architect, Backend, QA, DevOps, Data Engineer) producing plans, minimal-diff patches, property-based tests, manifests and migrations; and a four-agent Self-Healing Loop (Sentinel, Pathfinder, Synthesiser, Validator) detecting faults, walking the Neo4j [3] dependency graph towards root causes, and shadow-executing every patch in a locked-down Docker [10] sandbox, the whole pipeline orchestrated as a durable, replayable Temporal workflow [2].
2. **Severity-routed human approval.** An Approval Gate classifies each validated patch as HIGH (mandatory human review, for example schema changes, security-sensitive paths, or an agent's contract violation), MEDIUM (a 120-second intervention window, then auto-approval), or LOW (auto-approval for trivial UI-only diffs); Approve, Reject and diff-editing Modify decisions are all persisted as feedback.
3. **Real GitOps closure.** Approval opens a genuine GitHub pull request through a GitHub App, and a post-deploy probe watches for fresh fatal errors, invoking Argo CD rollback [6] to the prior known-good revision whenever recovery policy demands it.
4. **A Docker preview-deploy engine** that detects a cloned repository's stack, generates a missing Dockerfile, builds and runs the container under strict resource limits, health-checks it over HTTP and returns a live preview URL, feeding its failures back as first-class incidents.
5. **Intelligent role recommendation.** A heuristic engine over audit-log and login signal recommends privilege escalations and downgrades, each carrying an evidence-backed rationale that can be accepted or dismissed.
6. **A multi-tenant platform around the loop:** PostgreSQL Row-Level Security (RLS) [7], Role-Based Access Control (RBAC), MFA and SSO through WorkOS [12], Stripe billing [13], integrations with GitHub, Slack [17], Sentry [14], Datadog [16] and PagerDuty [15], a tamper-evident audit log, and an administrative console.

1.5 Scope of the Project

Under RBAC, Nexis serves three human roles:

- **Owner/Administrator** — provisions the organisation, connects integrations, sets recovery policy, decides on HIGH-severity approvals, manages members, billing and API keys, and acts on role recommendations.
- **Engineer/Member** — works daily with projects and incidents, watches live pipeline runs, and exercises Approve/Reject/Modify.

- **Viewer** — read-only visibility into incidents, recovery outcomes and cost, enforced through RBAC at the console level.

Beyond these roles, Nexis makes a deliberate departure from conventional systems analysis: the **nine agents are modelled as a first-class actor category** rather than as internal implementation detail. In a typical information system every actor in the use-case model is human, whereas here the Sentinel raises incidents, the Architect plans, the Backend agent writes code, and the Validator renders verdicts, each initiating and completing use cases as a human actor would, under the same audit and policy constraints. Treating agents as actors is precisely what lets the human roles shrink to supervisory decisions, and the framing carries through the requirement analysis and design chapters consistently.

The delivered scope is the platform described above, evaluated end-to-end against a locally hosted Large Language Model through Ollama [20]. The `nexis.sh` domain has been registered, but a credentialed public production deployment falls outside this report's scope, as the limitations discuss honestly.

1.6 Significance of the Project

The significance of Nexis lies chiefly in closing a loop that existing tools leave open. Detection without repair produces pager fatigue; code generation without validation produces risk; and automation without a human gate produces distrust that is entirely justified. Every individual stage of the lifecycle has been automated somewhere before; what has been missing is a system in which detect → diagnose → repair → validate → approve → deploy → verify runs as one continuous, durable and auditable workflow, with the human reduced from the transport mechanism between stages to a single, well-placed decision.

This report does not argue that claim in the abstract, but presents live evidence for it instead. Chapter 6 reports two genuine end-to-end recovery runs, both executed against a real, locally hosted language model [20]. In one, the approval window elapsed and the pipeline correctly rejected the patch on timeout, the safety mechanism working exactly as designed; in the other, a fault travelled the entire loop, from detection to an approved and deployed patch, in under two minutes, with per-agent token and cost figures recorded at every step. The same chapter also presents a live, HTTP-verified run of the preview-deploy engine, building and serving a real public repository that contained no Dockerfile to begin with.

Chapter 2: Literature Review

Nexis is assembled from tools that already exist and are already trusted in production, so the review that follows is written to separate two things that are easily confused: what the project *depends on*, and what it *extends*. This chapter therefore surveys the families of work the platform touches — Large Language Model (LLM) coding assistants, durable workflow orchestration, graph and causal root-cause analysis, and the self-healing/AIOps ecosystem — and states, tool by tool, the exact role each one plays inside Nexis. Section 2.1 presents that survey, Section 2.2 isolates the gap that none of the surveyed systems closes, and Section 2.3 states what the project claims as its own contribution against that gap.

2.1 Overview

Nexis is, at its core, a systems-integration project: its contribution comes not from a single new algorithm but from composing several established, well-engineered tools into a closed loop that none of them forms alone. Since intellectual honesty demands a clear account of what each tool already does, the discussion below goes tool by tool, distinguishing plain dependency from genuine extension.

Large Language Model (LLM) coding assistants. GitHub Copilot [1] is the most widely deployed Artificial Intelligence (AI) pair-programming tool: it completes code and, in its agent modes, drafts multi-file changes inside an editor session. Yet all of this happens inside the developer’s authoring context. Copilot never observes a production incident, never checks its own output against a failing system, and never deploys anything. Nexis’s Backend agent occupies that same “generate a minimal code change” niche, though what invokes it is a fault-detection pipeline rather than a keystroke, and its output is machine-validated before any human sees it.

Durable workflow orchestration. Temporal [2] provides durable, replayable workflow execution with automatic retry and state persistence. Nexis takes Temporal as-is, as a dependency: the entire nine-agent RecoveryPipeline runs as a single Temporal workflow, which is why every agent step survives a process restart and is recorded as a durable ActivityEvent. What Temporal does not provide is the engineering semantics layered on top: role routing, contract-violation detection, and severity classification are all Nexis’s own logic.

Graph databases for dependency analysis. Neo4j [3] is a native graph database, and Nexis stores its code-dependency graph inside it. The Pathfinder agent walks that graph from a fault symptom to candidate root-cause nodes, forwarding hop distance and node degree as evidence. Neo4j supplies storage and traversal, nothing more; the evidence-ranking applied to the traversal results is Nexis’s own code.

Causal inference tooling. DoWhy [4], Microsoft Research’s library for explicit causal modelling with refutation tests, is the methodological reference point behind Nexis’s causal-inference sidecar. It must be stated plainly, however, that Nexis does *not* fit a DoWhy structural causal model, since no interventional production data exists to fit one against. As such, the sidecar implements a documented proxy instead, ranking candidates by evidence-chain specificity, graph-structural proximity, textual overlap, and a scenario prior, with a transparent per-component confidence breakdown.

Observability standards. OpenTelemetry [5] standardises traces, metrics, and logs; Nexis adopts it, unmodified, as its instrumentation layer, exporting to Prometheus, Loki, and Tempo behind Grafana. It is a straightforward dependency: Nexis adds nothing to the standard itself.

GitOps deployment. Argo CD [6] reconciles a cluster against a Git repository and exposes Sync and Rollback operations. Nexis’s DevOps agent emits Argo CD manifests, and whenever a policy-gated Service Level Objective (SLO) breach is detected, the post-deploy rollback probe calls Argo CD’s real Rollback client. That is integration on Nexis’s part, not an extension of Argo CD itself.

Container runtimes. Docker [10] is the target Application Programming Interface (API) for two components inside Nexis: the Validator’s sandbox, a network-isolated, read-only, capability-dropped shadow execution of every candidate patch, and the preview-deploy engine. Podman [11] stood in for Docker in the evaluation environment, as a Docker-API-compatible runtime, and behaved identically; the substitution is reported honestly here, not hidden.

Property-based testing. Hypothesis [9] generates test inputs from declared properties. Nexis’s Quality Assurance (QA) agent puts it to a genuinely new use, auto-generating pytest/Hypothesis suites from the incident and the Backend diff while Hypothesis itself stays unmodified. A background QA loop then replays the stored suites and raises a fresh incident on any pass-to-fail regression.

Data lineage. OpenLineage [8] defines an open specification for run-level lineage events. Nexis’s Data Engineer agent emits OpenLineage-compatible RunEvents for every migration it proposes or applies; here too, Nexis conforms to the specification rather than extending it.

The incident-response ecosystem. Sentry [14], Datadog [16], and PagerDuty [15] respectively capture application errors, aggregate observability data, and route alerts to an on-call human. Nexis integrates all three as webhook-driven incident *sources* feeding the same Sentinel pipeline. The Slack API [17] is used in the opposite direction: the Approval Gate pushes a decision into Slack, where an engineer can approve or reject a validated patch with one click. A further set of platform dependencies is used exactly as documented, without modification: WorkOS for Single Sign-On (SSO) [12], Stripe for billing [13], PostgreSQL Row-Level Security (RLS) for tenant isolation [7], pgvector for knowledge-base retrieval [19], and Ollama [20], the local Large Language Model provider behind this report’s live evidence. Table 2.1 sets out this dependency-versus-contribution boundary.

Table 2.1: Position of surveyed tools relative to Nexis.

Tool	What it provides	Relation to Nexis
GitHub Copilot [1]	Editor-time code generation	Comparable to one agent (Backend) only
Temporal [2]	Durable workflow execution	Dependency; hosts the RecoveryPipeline
Neo4j [3]	Graph storage and traversal	Dependency; Pathfinder walks it
DoWhy [4]	Structural causal models	Reference point; proxy method used instead
OpenTelemetry [5]	Telemetry standard	Dependency; instrumentation layer
Argo CD [6]	GitOps sync/rollback	Integrated by DevOps agent and rollback probe
Docker [10] / Podman [11]	Container runtime	Dependency; sandbox and deploy engine
Hypothesis [9]	Property-based testing	Novel use: Quality Assurance agent generates suites
OpenLineage [8]	Lineage specification	Conformance by Data Engineer agent
Sentry / Datadog / PagerDuty [14, 16, 15]	Error/alert capture	Webhook incident sources
Slack API [17]	Team messaging	One-click approval channel

2.2 Research Gap

Each tool surveyed above is strong within its own boundary, and several of them are routinely chained together in industry: Sentry alerting into PagerDuty, an on-call engineer opening an editor with GitHub Copilot, Argo CD

eventually deploying the merge that follows. Moreover, what no single tool, nor any combination in common use, provides is a system that (a) models a software engineering *team* with explicit role specialisation, and (b) closes the loop from automatic fault detection to a deployed, verified fix, with human judgement concentrated at one severity-routed decision point.

This gap shows up at every seam of the conventional tool chain. GitHub Copilot generates code but never sees a production incident; it has no notion of a fault, a blast radius, or a deployment [1]. Observability platforms such as Datadog and Sentry detect and display faults with great fidelity, yet they never generate a candidate fix, let alone validate one [16, 14]. PagerDuty routes a human to the problem, but the repair itself stays entirely manual [15]. Argo CD deploys whatever a repository contains, though it authors nothing [6]. Temporal can durably orchestrate any of these steps; still, it carries no engineering semantics of its own [2]. In every case the loop breaks at the same place: between “a fault is known” and “a validated fix exists,” a human performs the unaided diagnosis, authoring, testing, review, and release.

Nexis closes precisely that span, end to end. Sentinel detects a fault using a statistical-process-control rule, an Exponentially Weighted Moving Average (EWMA) baseline with a mean-plus-three-sigma breach condition, rather than a fixed threshold. Pathfinder then walks the real Neo4j dependency graph to candidate root causes, which the causal-inference sidecar ranks with a transparent confidence breakdown. The Synthesiser classifies the scenario and selects which Execution Team agents are actually needed, and the workflow enforces that routing by visibly skipping the agents not selected. The Architect, for its part, produces a structured plan whose `affected_files` contract is mechanically checked against the Backend agent’s actual diff; a violation forces HIGH severity so the change can never auto-deploy unreviewed. Quality Assurance generates property-based tests from the incident and the diff [9], and the Validator then executes the patch and those tests inside a locked-down container sandbox, so no unvalidated patch ever reaches a human. Only at this final stage does the single human decision point engage, and only when severity warrants it: HIGH-severity changes require explicit review, MEDIUM-severity changes auto-approve after a 120-second window unless overridden, and trivial LOW-severity changes pass immediately. The decision itself is richer than a binary gate, since the engineer may Approve, Reject, or Modify the diff before it ships, with every terminal decision persisted for future preference learning. On approval, a real GitHub pull request is opened, and a post-deploy rollback probe watches for the regression that follows [6].

Two further properties set this apart from a merely scripted pipeline of existing tools. It may be noted, first, that role specialisation here is structural, not cosmetic: the five Execution Team agents (Architect, Backend, Quality Assurance, DevOps, Data Engineer) and the four Self-Healing Loop agents (Sentinel, Pathfinder, Synthesiser, Validator) have distinct inputs, outputs, and contracts, and the Architect’s plan constrains the others. Second, the loop closes over the platform’s own infrastructure too: when Nexis’s preview-deploy engine fails to build or health-check a repository, that failure is inserted as a first-class incident into the very same pipeline, and “write a working Dockerfile” becomes an ordinary repair task. To the author’s knowledge, no tool in common use today combines these properties in a single system.

2.3 Conclusion

The landscape surveyed in this chapter supplies every ingredient Nexis needs: durable orchestration [2], graph traversal [3], sandboxed execution [10, 11], property-based testing [9], GitOps delivery [6], and a mature incident-capture ecosystem [14, 16, 15]. What it does not supply is the loop between detection and repair; that is left open, for a human to close. Hence, Nexis’s contribution lies not in any single new algorithm but in the demonstrated composition of these tools into a role-specialised multi-agent team that detects, diagnoses, repairs, validates, and deploys autonomously, engaging human judgement once, at a severity-routed gate. How that composition was specified, designed, implemented, and tested is the subject of the chapters that follow.

Chapter 3: System Analysis and Requirements

Requirements for Nexis were derived from its actors rather than from a feature wish-list, because the platform’s defining property is *who* is allowed to act and *under what control*. This chapter therefore begins with the actors at the system boundary, three human roles and one autonomous nine-agent fleet (Section 3.1), states what each of them may do (Section 3.2), sets out the qualities the platform must hold while they do it (Section 3.3), and closes by testing the design for technical, economic, and operational feasibility against what was actually built (Section 3.4). Every capability granted in Section 3.2 is deliberately paired with a control in Section 3.3, so the requirements and the safety argument are read together rather than separately.

3.1 Requirement Analysis

Nexis was built as a role-based, multi-tenant platform in which every datum belongs to exactly one organisation, and every request is evaluated against the caller’s organisation and role. Given this, the requirement analysis begins naturally with the actors sitting at the system boundary. Three human actor types were identified:

- **Owner/Administrator** — owns membership, roles, billing, policy, integrations, and the final human decision on high-stakes automated repairs.
- **Engineer/Member** — investigates incidents, exercises the evaluation tooling, works the recovery pipeline daily.
- **Viewer** — a read-only observer (stakeholder or auditor): inspects system state, never changes it.

The role model itself is *additive*: each tier holds every permission of the tier below it, so Section 3.2 lists only what each role *adds*. (Internally, this maps to `owner/admin/member`; Viewer is RBAC-gated on the frontend and API.)

A fourth actor here is not human at all: the **nine-agent self-healing fleet**. Most requirement analyses enumerate only human actors, since in a conventional system every state change traces back to a person clicking something; that assumption does not hold here. The fleet acts autonomously inside the system boundary, detecting faults without being prompted and writing directly to the console’s own PostgreSQL, Neo4j, and object-storage stores, and its actions carry real external consequences: GitHub pull requests, Argo CD rollbacks [6], preview redeployes. Roughly half of all incident, patch, and audit-table writes are agent-authored, so leaving the fleet out would misrepresent the system; modelling it instead makes the safety analysis explicit, since every Section 3.2.4 capability is bounded by a Section 3.3 control — sandboxing, severity classification, or the human approval gate.

Level 0/1 use case and data flow diagrams for all four actors appear in Sections 4.6–4.7.

3.2 Functional Requirements

Functional requirements are stated per actor and, for the three human roles, *additively*: each subsection lists only what its actor gains over the tier beneath it, so an Owner/Administrator holds everything in Sections 3.2.1 through 3.2.3 and a Member holds everything in Sections 3.2.2 and 3.2.3. The fourth subsection is different in kind, since it describes an actor that acts without being asked; it is written as a list of autonomous capabilities, each bounded by one of the controls in Section 3.3.

3.2.1 Owner/Administrator can:

- Manage organisation membership: generate invite codes, send email invites, and remove members.
- Assign and change member roles, and act on the intelligent role-recommendation banner on the Members settings page: accept a recommendation (applying the role change under an optimistic-concurrency guard) or dismiss it (starting a 30-day per-rule cool-down).
- Edit organisation settings (name, profile, and workspace-level configuration).
- Manage billing through Stripe [13]: view usage, manage payment methods, and review invoices produced by the usage-ticker and invoice-roller cron jobs; set the organisation’s daily token budget on the cost tracker.

- View and export the append-only, hash-chained audit log (CSV export) and invoke its cryptographic verification endpoint to prove the chain has not been tampered with.
- Decide pending approvals from the recovery pipeline: **Approve** a validated patch, **Reject** it, or **Modify** it by editing the unified diff in place before it ships, with the edited diff persisted as a Reinforcement Learning from Human Feedback (RLHF) feedback example.
- Configure each project’s recovery policy, including whether the platform rolls back automatically on a Service-Level Objective (SLO) breach.
- Operate the Docker preview-deploy engine from a project’s *Ops* tab: trigger Deploy and Stop, watch live status, open the returned preview Uniform Resource Locator (URL), inspect build and container logs, and review deployment history.
- Install and manage integrations: the GitHub App (installation, repository listing, webhook-driven incident creation on merged pull requests), Sentry [14], Datadog [16], and PagerDuty [15] webhook sources, Slack [17] (OAuth install plus interactive one-click approve/reject directly from a Slack message), and Argo CD [6].
- Export evaluation-harness results as CSV (an administrator-only action), and receive the daily administrator digest email summarising the last 24 hours of incidents, repairs, approvals, and Mean Time To Recovery (MTTR).
- Monitor platform operations pages: System Health (parallel dependency probes of PostgreSQL, Redis, Neo4j, MinIO, and Temporal), webhook activity, and integration connection health.

3.2.2 Engineer/Member can:

- Sign up and sign in with email and password (a JSON Web Token (JWT) session in an HttpOnly cookie), sign in passwordlessly via magic link, or sign in through WorkOS Single Sign-On (SSO) [12]; recover access through the password-reset flow (hashed, time-limited reset tokens).
- Enrol Multi-Factor Authentication (MFA): Time-based One-Time Password (TOTP) with Quick Response (QR) code provisioning.
- Manage their own active sessions: list signed-in devices with Internet Protocol (IP) address, user agent, and last-seen time (the caller’s own session carries a “This device” badge), and revoke any session remotely.
- Create, list, and revoke scoped API keys (`nx_live_...` bearer tokens) for programmatic access.
- Create projects through the new-project wizard and connect them to GitHub repositories.
- Browse the incidents list and open an incident’s detail page, where the nine-agent pipeline timeline streams live over Server-Sent Events (SSE), including per-agent status, token counts, cost, and structured payloads such as the Backend agent’s raw generated diff.
- Trigger fault-injection runs from the Live-Demo console page, which presents 27 curated scenario cards drawn from a broader fixture-scenario taxonomy; only one scenario is fully wired end-to-end at present, and the remainder are labelled “Coming soon”.
- Run the evaluation harness: side-by-side OpenAI-versus-Ollama provider comparisons over fixture incidents, with persisted transcripts and cost figures; query the pgvector-backed knowledge base [19] and view its retrieval status.
- Complete the post-incident NASA Task Load Index (NASA-TLX) workload survey [18], the platform’s instrument for measuring operator cognitive load.
- Use the operational consoles: the agents fleet page and per-agent detail views, the workflows list, the activity stream, the validator sandbox page, and the recovery-pipeline view.

3.2.3 Viewer can:

- Sign in and view the dashboard: KPI tiles (MTTR, recovery success rate, open incidents, mean tokens per run), the incidents-over-time chart, the system status panel, the projects health grid, and the recent activity feed.
- Read the incidents list and incident detail timelines, including the live SSE stream, without any decision or mutation controls.
- Read project pages (overview, activity, and status), the performance page, and the agents, workflows, and activity-stream pages in read-only form.
- Take no mutating action of any kind: no approvals, no deployments, no settings changes, and no key or session management beyond their own account.

3.2.4 The Nine-Agent Fleet autonomously:

- **Detects** faults (Sentinel): an always-on detector applies two rules, a per-row fatal-level trigger and a statistical-process-control spike rule that maintains a per-organisation Exponentially Weighted Moving Average (EWMA) mean and variance baseline and fires on a breach of $\text{mean} + 3\sigma$, with a Poisson floor for low-count noise and an absolute-floor warm-up fallback.
- **Diagnoses** root causes (Pathfinder): walks the Neo4j code dependency graph [3] outward from the fault symptom and forwards candidate root-cause nodes, annotated with hop distance and degree, to the causal-inference sidecar, which ranks candidates by evidence-chain specificity, structural proximity, textual overlap, and a scenario prior, returning a per-component confidence breakdown [4].
- **Classifies and routes** (Synthesiser): assigns the incident a scenario, using a regular-expression fast path with an LLM fallback, and computes an ordered `selected_agents` list; the workflow enforces this routing, skipping non-selected execution agents with a visible “skipped” timeline state.
- **Plans** the repair (Architect): decomposes the incident into `plan_steps`, `affected_files`, and a `risk_level`, and serves as a contract other agents may not silently violate: if the generated diff touches a file outside `affected_files`, the workflow flags a contract violation and forces HIGH severity, so the patch can never auto-deploy unreviewed.
- **Codes** the fix (Backend): generates an LLM-guided minimal unified diff, the smallest valid change rather than a full-file rewrite.
- **Tests** the fix (QA): auto-generates property-based test suites (pytest/Hypothesis [9]) from the incident and the diff; a background continuous-QA loop replays previously generated suites against the sandbox and raises a fresh incident on any pass-to-fail regression, closing a second, independent loop.
- **Packages** the deployment (DevOps): emits CI/CD and Argo CD manifests [6] and drives post-deployment health monitoring.
- **Handles data changes** (Data Engineer): proposes paired forward and reverse SQL migrations, runs a proactive schema-drift detector that introspects `information_schema` against a captured baseline and raises a real incident on drift, and emits OpenLineage-compatible run events [8] for every migration proposal and application.
- **Validates** every candidate patch (Validator): executes it in a locked-down Docker sandbox [10] (no network access, a read-only filesystem, all capabilities dropped, a process limit, and `no-new-privileges`), runs the QA-generated property tests, and reports pass/fail and coverage; no unvalidated patch ever reaches the approval gate.
- **Classifies severity** at the approval gate: HIGH for schema-drift scenarios, sensitive-path matches (SQL, migrations, authentication, security, and cryptography paths), unknown scenarios, or contract violations; MEDIUM otherwise, with a 120-second auto-approve countdown unless a human intervenes; LOW (tiny UI-only diffs) auto-approves immediately.
- **Deploys** approved patches (GitOps): opens a real GitHub pull request through a GitHub App installation (blob → tree → commit → ref → pull request); when the triggering incident originated from the preview-deploy engine, a successful merge additionally triggers an automatic redeploy.
- **Rolls back** on regression: a post-deploy probe watches for a fresh fatal incident on the same organisation and service within a bounded window and, when project policy permits, invokes Argo CD’s rollback client to restore the prior known-good revision.
- **Records** every step as a Temporal-durable activity event [2], streamed live to the incident detail page over Server-Sent Events, and **feeds the deploy engine’s failures back into itself**: a preview deployment that fails to build or health-check files a real incident (`source: "deploy_engine"`) into the same Sentinel-to-pipeline path as every other source, making a broken build a self-healing target like any production fault.

3.3 Non-Functional Requirements

Functional capability is only half the specification; the properties below are the ones the platform must hold *while* exercising it, and each is realised by a concrete mechanism rather than by convention. Table 3.1 summarises them, and the paragraphs that follow give the reasoning and the rejected alternatives.

Table 3.1: Non-functional requirements and realisation.

Requirement	Realisation in Nexis
Multi-tenancy	PostgreSQL Row-Level Security (RLS) on essentially every table [7]
Durability	Temporal-orchestrated workflows survive crashes and restarts [2]
Security	Sandboxed patch execution, encrypted patch storage, hashed tokens, hash-chained audit log
Scalability	Stateless Go services, queue-decoupled workers, parallel dependency probes
Responsiveness	Server-Sent Events stream the live incident timeline
Usability	Role-scoped console, one-click approvals, plain-language rationales

Multi-tenancy and isolation. Tenant isolation cannot be left to application code remembering to add a `WHERE org_id = ...` clause every time, since that approach fails the moment one query is missed. Nexis enforces isolation inside the database itself, through PostgreSQL Row-Level Security policies [7]: a request-scoped transaction binds `app.current_org_id`, and the database refuses to return or mutate rows belonging to any other organisation, whatever the application layer asks for. This holds uniformly for human requests and for agent writes.

Durability of long-running work. A recovery run can span many minutes, crosses several services, and often waits on a human decision, so it must survive crashes and restarts without losing its place. All pipeline state therefore lives in Temporal [2], where every agent step is a journaled activity and a resumed worker replays deterministically to exactly where it stopped. An in-memory orchestrator was considered and rejected: a single crash mid-repair would strand an incident in an unknown state.

Security. Autonomously generated code is treated as untrusted by construction. Candidate patches execute only inside a Docker sandbox [10], with networking disabled, a read-only filesystem, all Linux capabilities dropped, a process-count limit, and `no-new-privileges`. Patch blobs are encrypted in object storage, and password-reset and API tokens are stored only as hashes (reset tokens are 32-byte random values hashed with SHA-256, carrying a 30-minute time-to-live and a non-leaky 202 response). The audit log is append-only and hash-chained, so tampering with a past entry breaks the chain and is detectable through the verification endpoint.

Scalability and responsiveness. The control plane, validator, GitOps, and deploy-engine services are independent, stateless Go modules that scale horizontally on their own; Temporal decouples workflow load from any single process, Redis handles rate limiting and caching, and the System Health page’s dependency probes fan out in parallel under a bounded timeout, so one slow dependency cannot stall the page. An engineer watching a live repair, meanwhile, needs agent transitions as they happen rather than after a polling delay, so the incident detail page subscribes to a Server-Sent Events stream and each Temporal-recorded event reaches the browser within moments; the live runs in Chapter 6 were observed exactly this way.

Usability. The console has to make a genuinely complex system operable by an ordinary engineer with a browser. Role-scoped navigation shows each person only what they can act on, approvals reduce to a one-click decision with the full diff in view, severity and countdown state stay visible throughout, and role recommendations carry a plain-language rationale naming the concrete evidence behind them.

3.4 System Feasibility

Feasibility is argued here against the delivered system rather than against a proposal, since the platform described in Chapters 4 to 6 was built, run, and measured before this section was written. Three questions are answered in turn:

whether the technology choices can carry the design (Section 3.4.1), whether the system can be built and operated at a defensible cost (Section 3.4.2), and whether the people who must live with it can actually do so (Section 3.4.3).

3.4.1 Technical Feasibility

Nexis was deliberately built as a polyglot system. The four independent backend services are written in Go, for static binaries, cheap concurrency, and a first-class Docker API client on which the webhook and container work depends. The console, however, is Next.js/React/TypeScript, chosen for its role-gated, SSE-streaming, Monaco-editing interface, while the causal sidecar runs on Python/FastAPI, on the scientific-Python/Hypothesis [9] stack. PostgreSQL 17 with pgvector [19] supplies transactions, RLS, and vector retrieval in one engine, whereas Neo4j [3] holds the dependency graph because diagnosis is graph traversal, which relational queries express poorly. Temporal [2] was proven by both live recovery runs in Chapter 6, and the whole system was built and verified on one development machine, with Podman [11] substituting for Docker [10].

3.4.2 Economic Feasibility

Every dependency Nexis relies on is open source, namely PostgreSQL, Neo4j Community, Redis, MinIO, Temporal, OpenTelemetry [5], and Grafana, so there is no licence cost. The one significant cost is LLM inference, handled as a dual path: OpenAI models for production, local Ollama [20] models (`gpt-oss:20b`, `qwen2.5-coder:14b`) for development. Every live run in this report in fact used the local provider, so the complete system was exercised at zero token cost; the cost tracker and daily budgets bound the paid path.

3.4.3 Operational Feasibility

On logging in, administrators see organisation-wide KPIs, approvals, and system health; engineers see incidents and the live pipeline; viewers see the same picture, read-only. High-stakes autonomy, in this regard, reduces to a single click, Approve, Reject, or Modify, optionally from Slack [17], while severity classification lets low-risk fixes pass unattended but never the sensitive ones. The built-in NASA-TLX instrument [18] lets the operator-workload claim eventually be measured rather than simply asserted; the intended operator, furthermore, is an ordinary engineer with a browser, not a dedicated operations team.

Chapter 4: System Design and Architecture

The design of Nexis is presented across four complementary levels in this chapter: the structural view (Sections 4.1–4.5), the functional view through use case diagrams (4.6), the data view through Data Flow Diagrams (4.7), and a UML sequence diagram tracing one complete recovery run (4.8). Since the self-healing loop is the central contribution of the project, it has been given the most space of the four.

4.1 System Context

Figure 4.1 places Nexis at the centre of its environment. Three human roles sit around it (Owner/Administrator, Engineer/Member, Viewer), the nine-agent fleet acts as a fourth, autonomous actor, and a wider ring of external systems completes the picture: GitHub, Slack [17], the LLM provider (OpenAI or Ollama), Argo CD, Stripe [13], the alert sources Sentry [14], Datadog [16], and PagerDuty [15], WorkOS SSO [12], and SES/SMTP mail.

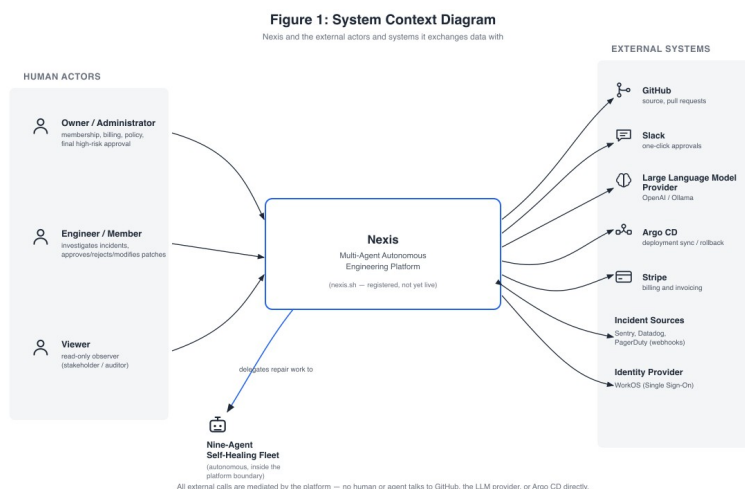


Figure 4.1: System Context Diagram. (Nexis and the external actors and systems it exchanges data with).

Every entry point, whether the web console, webhooks, Slack callbacks, or a plain HTTP client, converges on the control-plane; no human or agent talks to a data store or an external integration directly. It should be disclosed plainly that the `nexus.sh` domain is registered, but the platform is not yet publicly live (Section 4.5).

4.2 Component Architecture

Figure 4.2 opens up the platform boundary. The Next.js 16 web console sits to one side; moreover, the centre is anchored by a single Go control-plane (`:8080`) that hosts the chi-routed API layer (authentication, RBAC, RLS binding, rate limiting), the always-on Sentinel detector (EWMA $+3\sigma$ spike rule), and the durable Temporal RecoveryPipeline workflow [2], which orchestrates the nine-agent loop.

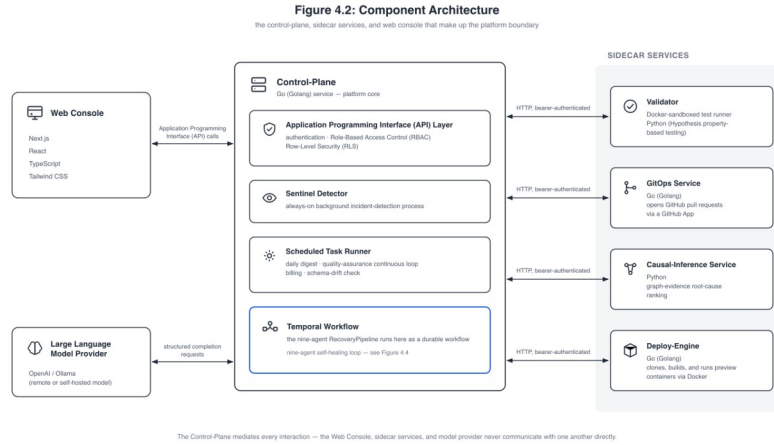


Figure 4.2: Component Architecture. (the control-plane, sidecar services, and web console that make up the platform boundary).

The control-plane also carries the scheduled task runner (daily digest, the continuous-QA loop, billing tickers, schema-drift detection), and the Temporal RecoveryPipeline it hosts orchestrates the nine-agent loop as one durable workflow: detect → diagnose → plan → design → code → test → package/migrate → approve (severity-routed) → deploy → verify → roll back if needed. Four bearer-authenticated sidecars sit around it, each independently deployable and reachable only from the control-plane:

- **Validator** (:8081) — the Docker-sandboxed test runner [10] that shadow-executes every candidate patch under `--network=none`, `--read-only`, and `--cap-drop=ALL`, running the QA-generated Hypothesis property tests [9];
- **GitOps service** (:8082) — GitHub App pull-request automation along the blob → tree → commit → ref → PR path;
- **Causal-inference service** (:8090) — the Python/FastAPI graph-evidence ranker scoring Pathfinder’s candidates by evidence-chain specificity, structural proximity, and textual overlap, an honest statistical proxy rather than a fitted structural causal model [4];
- **Deploy-engine** (:8091) — clone, stack detection, Dockerfile generation, build, and health-checked container run for preview deployments.

An Argo CD client module supplies the Sync and Rollback integration [6] used by the post-deploy SLO probe. The LLM spine, for its part, pairs OpenAI (gpt-4o family) with Ollama [20] (gpt-oss:20b, qwen2.5-coder:14b) as a fully supported alternative, backed by pgvector retrieval [19] that feeds code-context passages to the Architect, Backend, and QA agents, and by a structured-output stage that validates every completion against a per-agent JSON Schema and retries on mismatch. The mediation rule from Section 4.1 holds throughout: console, sidecars, and model provider never talk to one another directly.

4.3 Data and Storage Architecture

The four storage systems, along with Temporal’s workflow persistence layer, are laid out in Figure 4.3.

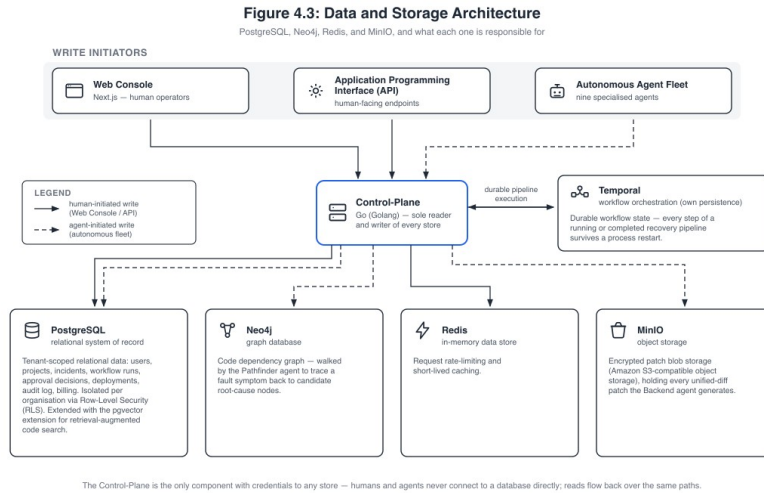


Figure 4.3: Data and Storage Architecture. (PostgreSQL, Neo4j, Redis, and MinIO, and what each one is responsible for).

PostgreSQL 17 with pgvector is the relational system of record, made multi-tenant via Row-Level Security [7] across 33 migrations; its key tenant-scoped tables (`users`, `org_members`, `projects`, `incidents_raw`, `workflow_runs`, `approval_decisions`, `deployments`, `lineage_events`, `audit_log`, `feedback_examples`, `role_recommendations`, `digest_reports`) reappear as the data stores of the Data Flow Diagrams in Section 4.7. Neo4j [3] holds each connected repository’s code dependency graph, which Pathfinder walks from symptom to candidate root cause; Temporal maintains its own persistence, so every step of a running or completed pipeline survives a process restart; Redis serves rate limiting and short-lived caching; and MinIO is the encrypted patch store, S3-compatible object storage holding every unified diff the Backend agent generates.

The diagram also separates the two classes of write initiator, human writes arriving through the console and API, and agent writes from the autonomous fleet, which is what makes the isolation claim in its footer meaningful: only the control-plane holds credentials to any store, neither humans nor agents ever connect to a database directly, and reads flow back over those same mediated paths.

4.4 Self-Healing Loop Pipeline

Figure 4.4 presents the central contribution of the project as a single flow, the nine agents running in order: Sentinel (detect), Pathfinder (localise via the Neo4j graph), Synthesiser (classify, select agents), Architect (plan and file-level contract), Backend (minimal patch), Quality Assurance (property-based tests), DevOps running in parallel with Data Engineer (manifests and migrations), and finally Validator (sandboxed shadow-execution).

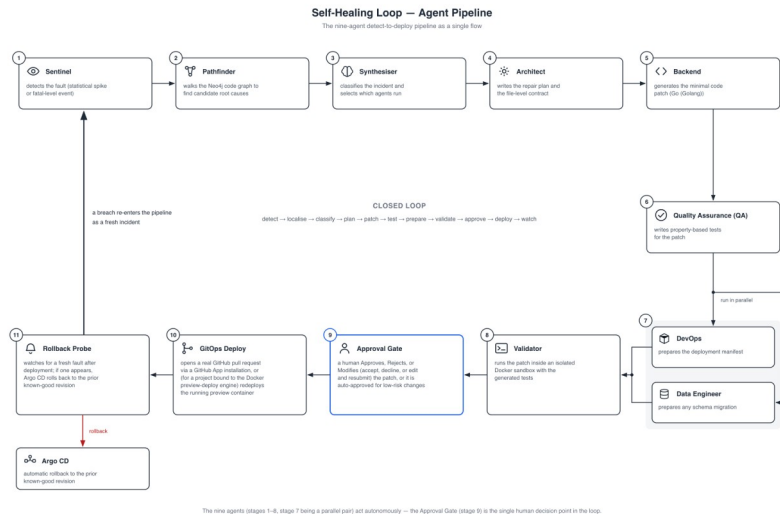


Figure 4.4: Self-Healing Loop — Agent Pipeline. (the nine-agent detect-to-deploy pipeline as a single flow).

A person enters the loop only at this point: the Approval Gate (approve, reject, or modify; auto-approved for low-risk changes) forms the single human decision point in the whole pipeline. GitOps Deploy consequently opens a real pull request through a GitHub App installation or, for a project bound to the preview-deploy engine, redeploys the running preview container, while the Rollback Probe watches for a fresh fault afterwards: should one appear, Argo CD rolls back to the prior known-good revision and the breach re-enters the pipeline as a fresh incident.

The closing arc of the figure captures the property that gives the project its title — detect → localise → classify → plan → patch → test → prepare → validate → approve → deploy → watch, with the watch stage feeding back into detection. Sections 4.7.3 and 4.8 decompose that same flow into its data stores and its message-by-message timing.

4.5 Deployment and Infrastructure

Figure 4.5 lays out how the platform is built, delivered, and observed: GitHub Actions handles CI, Docker (Podman-compatible) serves as the universal runtime, Argo CD carries out GitOps delivery, and Terraform declares the AWS infrastructure as code.

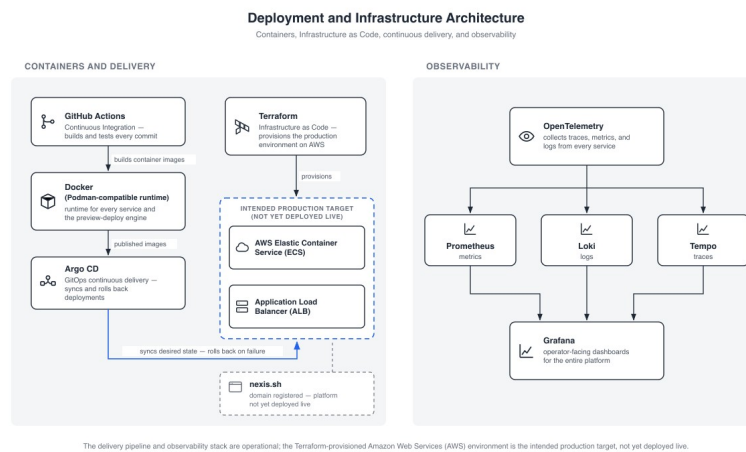


Figure 4.5: Deployment and Infrastructure Architecture. (containers, Infrastructure as Code, continuous delivery, and observability).

On the delivery side, GitHub Actions builds and tests every commit and produces the container images, Argo CD syncs desired state and rolls back on failure, and Terraform declares the production environment as code. This diagram, too, is deliberately honest about status: the AWS ECS cluster behind an Application Load Balancer is the *intended* production target declared in Terraform, not yet deployed live, and the registered `nexis.sh` domain does not yet serve the platform. Observability across all layers follows the OpenTelemetry pipeline [5], with traces, metrics, and logs collected from every service into Prometheus, Loki, and Tempo, all of it visualised in Grafana as the operator-facing dashboards. The container runtime boundary, finally, is the Docker Engine API, which Podman [11] satisfies identically; this was more than a theoretical claim, since the evaluation environment used for this report actually ran the Validator sandbox and the deploy-engine on Podman.

4.6 Use Case Diagrams

Where Sections 4.1 to 4.5 describe what the system *is*, the use case view describes what may be *done with it*, and it is presented at two levels of detail. Level 0 (Section 4.6.1) fixes the boundary and the four actors that sit against it; Level 1 (Section 4.6.2) opens that single bubble into the concrete use cases each actor holds, with generalisation arrows carrying the role-additive model of Section 3.2 into the diagram itself.

4.6.1 Use Case Diagram, Level 0

The coarsest functional view appears in Figure 4.6: the three human actors and the autonomous nine-agent fleet set against a single system boundary, with the mediated external systems placed to the right.

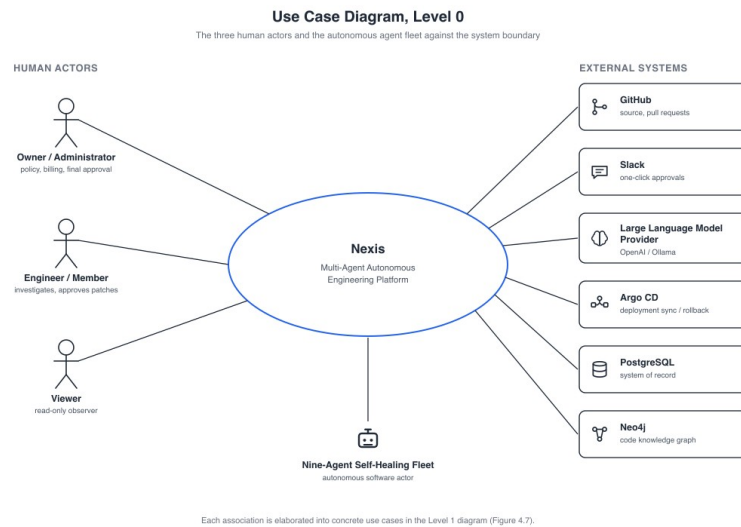


Figure 4.6: Use Case Diagram, Level 0. (the three human actors and the autonomous agent fleet against the system boundary).

Mediation is the essential property here: every association terminates at the system boundary, and no human ever calls GitHub, the LLM provider, or Argo CD directly. The agents act on the external systems on the humans' behalf instead, surfacing back to a person only at the approval gate.

4.6.2 Use Case Diagram, Level 1

Figure 4.7 expands the Level 0 bubble into concrete use cases, grouped by minimum role, where generalisation arrows make the role inheritance explicit.

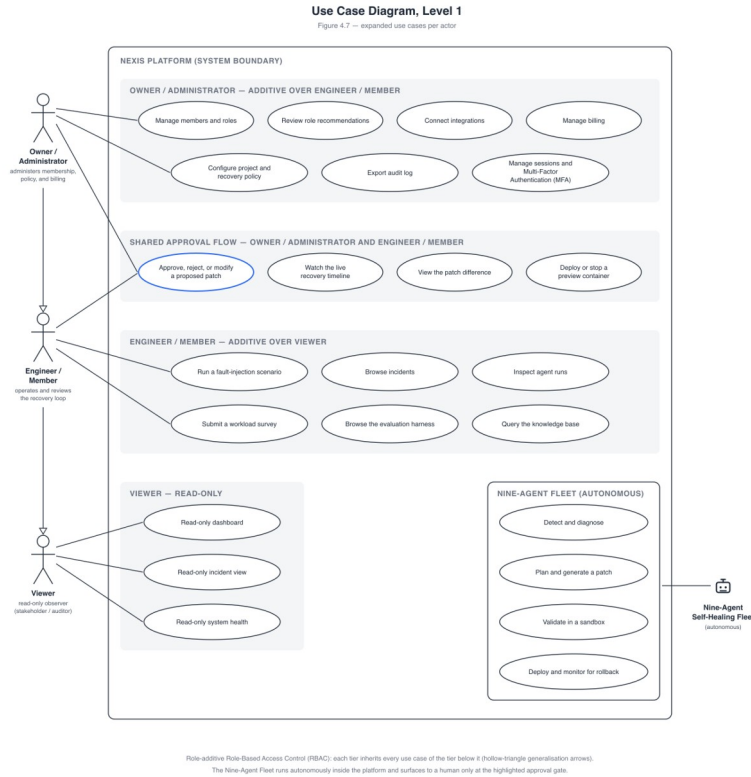


Figure 4.7: Use Case Diagram, Level 1. (expanded use cases per actor).

The administrative tier holds fifteen use cases in total (membership, role recommendations, integrations, recovery policy and kill switch, billing, the RLHF/JSONL feedback export, audit, MFA). Four approval-flow use cases are shared with the Engineer/Member, whose own tier includes demo scenarios, agent runs, and the NASA-TLX survey [18]; the Viewer holds exactly three read-only use cases. The model is role-additive, in that each tier inherits every use case of the tier below it. As such, the read-only Viewer experience is enforced through RBAC gating in the console, even though the database role enumeration itself is only `owner/admin/member`.

4.7 Data Flow Diagrams

The data view descends in the same way, and it is the view that carries the multi-tenancy argument, since every flow drawn here terminates in a store that Row-Level Security [7] scopes to one organisation. Section 4.7.1 gives the context level, a single process against its three top-level stores; Section 4.7.2 decomposes the administrative surface into ten numbered processes; and Section 4.7.3 decomposes the self-healing loop into twelve, ending at the rollback probe that feeds its own output back into detection.

4.7.1 Data Flow Diagram, Level 0 (Context)

At the context level, Figure 4.8 shows the entire platform as a single process 0 with three top-level stores: D1 the tenant database (PostgreSQL, under Row-Level Security), D2 the code graph (Neo4j), and D3 the encrypted patch store (MinIO).

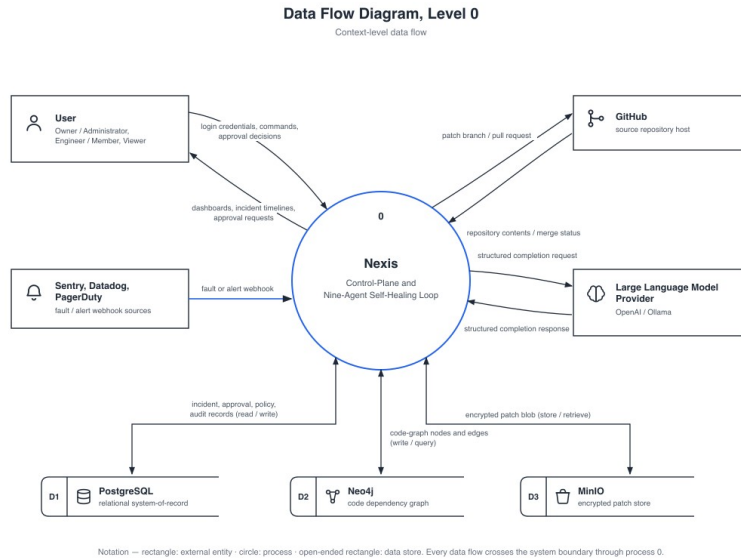


Figure 4.8: Data Flow Diagram, Level 0. (context-level data flow).

Four entity flows cross the boundary: the user, GitHub, the alert sources (producers of incidents, not consumers), and the LLM provider. Every one of these is mediated by process 0, since there is no direct path from a human to an integration.

4.7.2 Data Flow Diagram, Level 1 — Owner/Administrator

Figure 4.9 decomposes the administrative surface into ten numbered processes, spanning identity and access, platform administration, and incident and approval operations, all over the tenant-scoped PostgreSQL stores; Table 4.1 summarises the most essential of these.

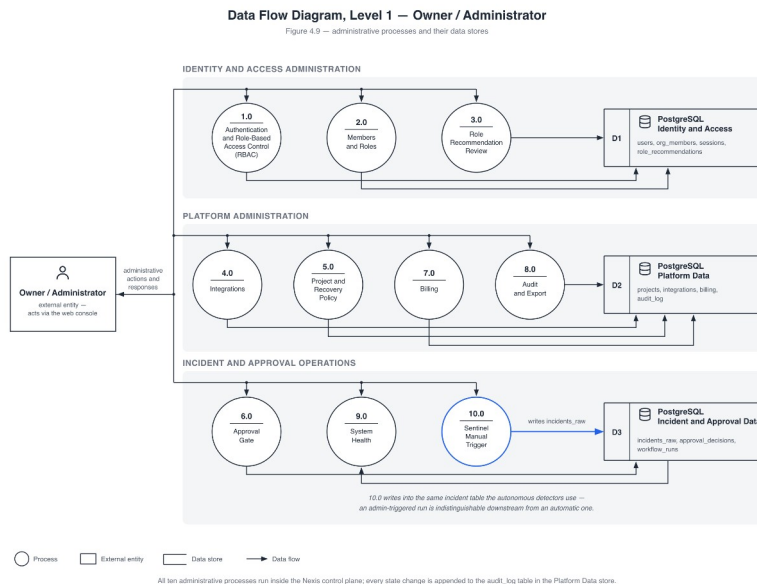


Figure 4.9: Data Flow Diagram, Level 1 — Owner / Administrator. (administrative processes and their data stores).

Table 4.1: Owner/Administrator Data Flow Diagram Level 1 processes and their primary data stores.

Process	Responsibility	Primary store written
1.0 Authentication and RBAC	login, JWT session issue, role checks	users, sessions
2.0 Members and Roles	invite, promote, demote, remove	org_members
3.0 Role Recommendation Review	heuristic promote/downgrade suggestions	role_recommendations
4.0 Integrations	connect GitHub / Slack / Sentry / Argo CD	integrations
5.0 Project and Recovery Policy	recovery policy, SLO, kill switch	projects
6.0 Approval Gate	approve / reject / modify decisions	approval_decisions
7.0 Billing	Stripe usage ticker, invoices, plan	billing tables (Stripe)
8.0 Audit and Export	hash-chained audit writes, CSV export	audit_log
9.0 System Health	parallel dependency probes (read-only)	— (no persistent write)
10.0 Sentinel Manual Trigger	administrator-initiated detection run	incidents_raw

Processes 1.0 to 3.0 form the identity and membership surface, with 1.0 also binding the request-scoped RLS context and owning session listing, revocation, and TOTP enrolment; 4.0 and 5.0 configure what the autonomous loop is allowed to act on, and 5.0 additionally carries the kill switch that suspends autonomous recovery for a project. Process 9.0 fans probes out to PostgreSQL, Redis, Neo4j, MinIO, and Temporal in parallel and persists nothing at all.

The most important flow, though, is 10.0: a manual detection run writes into the *same* incidents_raw table as the automatic detectors, with no separate code path, no parallel queue, and no privileged shortcut, so an administrator-triggered run is indistinguishable downstream from an automatic one. Hence every autonomous-path guarantee — durability, validation, severity routing, auditability — holds for manual runs as well, at no extra engineering cost, and every state change lands in the hash-chained audit_log.

4.7.3 Data Flow Diagram, Level 1 — Self-Healing Loop

Twelve numbered processes make up the loop in Figure 4.10's decomposition: detection and decision, the Layer-1 execution team, and the validation–approval–deployment–verification tail, spread over five stores (D1 incidents, D2 code graph, D3 patch store, D4 approvals and feedback, D5 deployments and lineage).

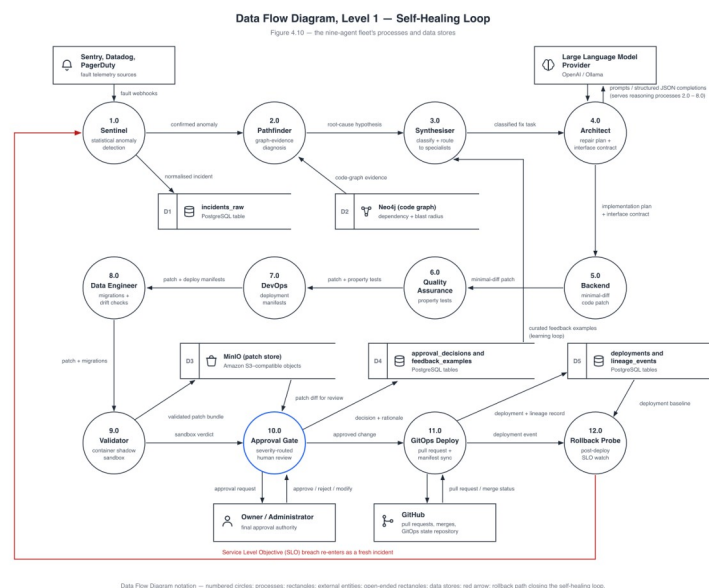


Figure 4.10: Data Flow Diagram, Level 1 — Self-Healing Loop. (the nine-agent fleet’s processes and data stores).

The closing annotation states the property that gives the project its very title: the loop is closed. An SLO breach seen by the rollback probe (12.0) lands in `incidents_raw` as a fresh fatal and re-enters at Sentinel (1.0), with an Argo CD rollback to the prior known-good revision whenever `RollbackOnSLOBreach` is set. Thus, recovery, verification, and re-detection all form one cycle over a single shared data path.

4.8 Self-Healing Loop Sequence Diagram

One complete recovery run is traced in Figure 4.11 across all nine agent lifelines, in eighteen numbered messages. Each message is a named Temporal activity, durably recorded and streamed to the incident timeline over Server-Sent Events, in the exact order that the Chapter 5 screenshots show.

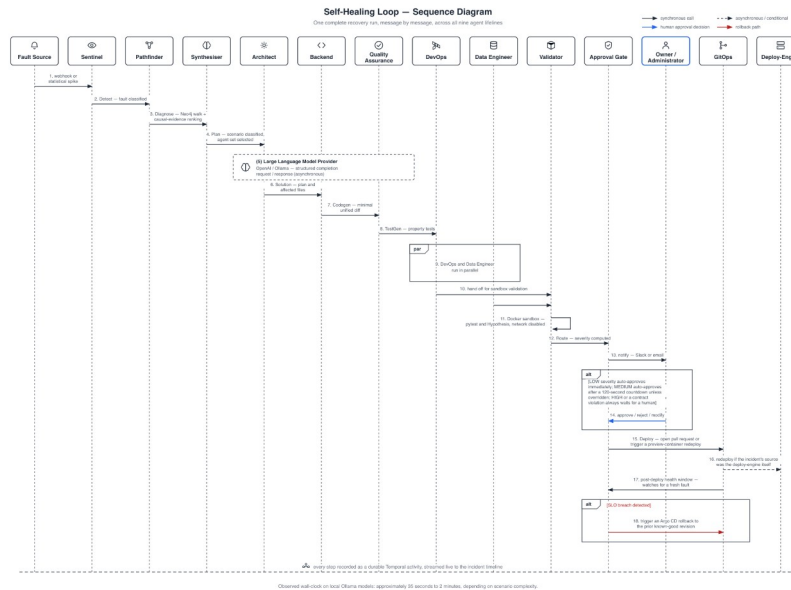


Figure 4.11: Self-Healing Loop — Sequence Diagram. (one complete recovery run, message by message, across all nine agent lifelines).

No unvalidated patch is allowed past the sandboxed Validator (`--network=none`, `--read-only`, `--cap-drop=ALL`, `pytest/Hypothesis suite [9]`). Severity routing then forces an explicit human decision for HIGH: schema drift, sensitive-path diffs (`*.sql`, `migrations/`, `auth/`, `security/`, `crypto/`), unknown scenarios, or a contract violation (a diff falling outside the Architect’s `affected_files`), whereas LOW auto-approves and MEDIUM auto-approves after 120 seconds. One of the two live runs in Chapter 5 genuinely timed out at this gate, the safety mechanism working as designed. Observed wall-clock time ran from approximately 35 seconds to 2 minutes on local Ollama models, a figure corroborated by Chapter 5’s fully successful run of 1 minute 40 seconds.

Taken together, these views describe one coherent design: a mediated boundary, a control-plane serving as the sole gateway to every component and store, a role-additive human surface set beside an autonomous fleet, a single shared data path for both manual and automatic operation, and a recovery cycle that feeds its own failures back into itself. Chapter 5 goes on to present the implementation, while Chapter 6 supplies the evidence that the loop actually closes in practice.

Chapter 5: Implementation

The system covered here is the one that actually runs, not a proposed version of it: the technology stack, a screen-by-screen tour of the platform, and the database schema underneath the multi-tenant control plane. Every screenshot is an unretouched capture from the running development environment, and errors or pending states are reported as they happened, not swapped for a tidier run.

5.1 Tools and Technology

Nexis is built as a polyglot pnpm + Turborepo monorepo, with each subsystem implemented in whichever language best suits its role:

- **Backend:** Go 1.25 powers four modules: `services/control-plane` (multi-tenant API), `services/validator` (sandboxed patch validation), `services/gitops` (GitHub-App deployment), and `services/deploy-engine` (Docker preview deploys). A Python/FastAPI sidecar, `services/causal-inference` (port 8090), ranks candidate root-cause nodes for the Pathfinder agent, while Temporal [2] durably orchestrates the nine-agent RecoveryPipeline, with every step logged as an ActivityEvent so crashed workers resume in place.
- **Frontend:** Next.js 16, React 19, and TypeScript make up `apps/web`, styled with Tailwind CSS 4 and Radix UI. Monaco handles unified-diff editing, Recharts renders the charts, and Server-Sent Events keep incident pages live without resorting to polling.
- **Data:** PostgreSQL 17 serves as the system of record (Row-Level Security (RLS) [7], 33 migrations); pgvector [19] backs knowledge-base retrieval; Neo4j [3] holds the code dependency graph; Redis handles rate limiting and caching; MinIO stores S3-compatible encrypted patch blobs; and Temporal keeps its own durable event history.
- **AI / LLM:** OpenAI (GPT-4o family) is the production-configured provider, while Ollama [20] (`gpt-oss:20b`, `qwen2.5-coder:14b`) is fully supported; in fact, all live-run evidence in this chapter used Ollama end-to-end, with no cloud Large Language Model (LLM) credentials involved. The Quality Assurance (QA) agent generates Hypothesis [9] property-based tests, which the Validator then executes inside its sandbox. It may be noted that the causal-inference sidecar is a statistical/graph ranking algorithm rather than a fitted DoWhy [4] model, since that would require interventional production data that does not yet exist, a limitation stated openly.
- **DevOps:** Docker API [10] serves both the Validator sandbox and preview deploys and is Podman-compatible [11]; in the evaluation environment, Podman stood in for it and worked identically. Terraform provisions AWS ECS behind a load balancer, OpenTelemetry [5] feeds Prometheus, Loki, Tempo, and Grafana, and Argo CD [6] backs the DevOps agent's Sync/Rollback client.
- **Version control:** Handled via Git and GitHub, with a GitHub App installation supplying short-lived tokens for cloning and for the GitOps blob → tree → commit → ref → Pull Request deployment path.

5.2 Project View

The captures that follow are ordered the way a new organisation would encounter the platform: authentication first, then the administrator console, then the self-healing loop itself, which carries the evidentiary weight of the report, followed by the preview-deploy engine and, last, the integration and observability surfaces.

5.2.1 Authentication and Onboarding

Access relies on a local JSON Web Token (JWT) session, kept in an HttpOnly cookie, with optional WorkOS [12] Single Sign-On available on top of it. The sign-up screen (Figure 5.1) creates the account and the sign-in screen (Figure 5.2) establishes the session, while the forgot-password flow (Figure 5.3) issues a 32-byte reset token, SHA-256-hashed with a 30-minute time-to-live; it returns the same 202 response regardless of whether the e-mail address actually exists, hence no account information leaks out.

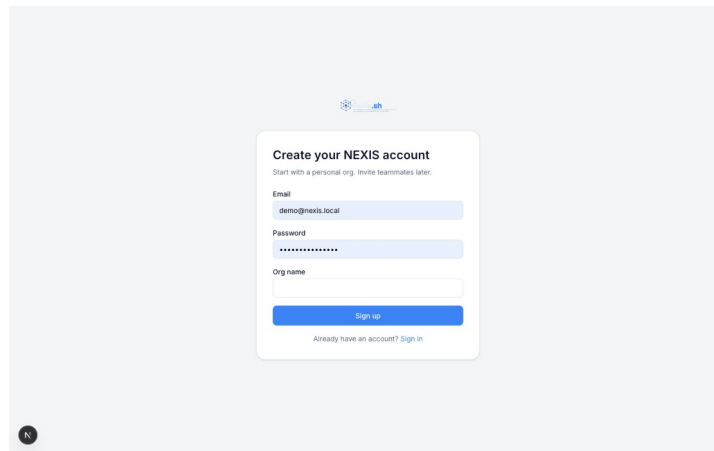


Figure 5.1: Sign-Up Screen. (new account registration).

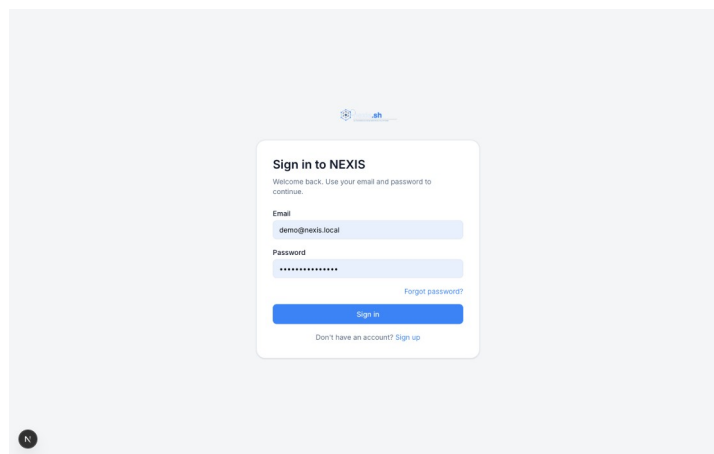


Figure 5.2: Sign-In Screen. (JWT session login with HttpOnly cookie).

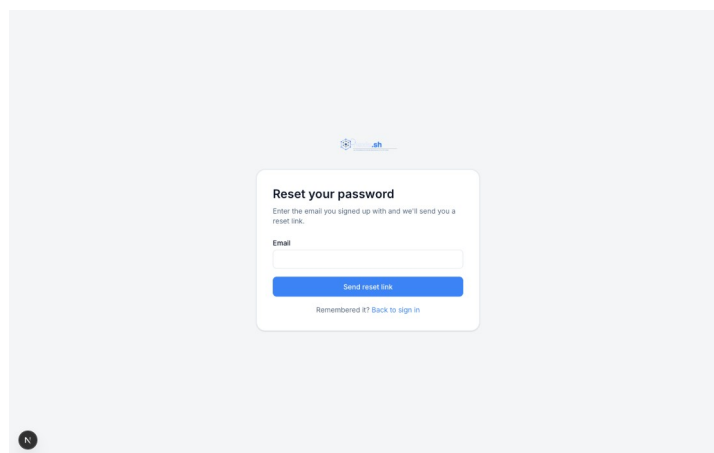


Figure 5.3: Forgot-Password Screen. (non-leaky password-reset request).

5.2.2 Administrator / Owner Console

Once the administrator logs in, the console home (Figure 5.4) presents KPI tiles covering Mean Time To Recovery (MTTR), recovery success rate, open incidents, and mean tokens per run; alongside these sit an incidents-over-time chart, a system status panel, a projects health grid, and an activity feed.

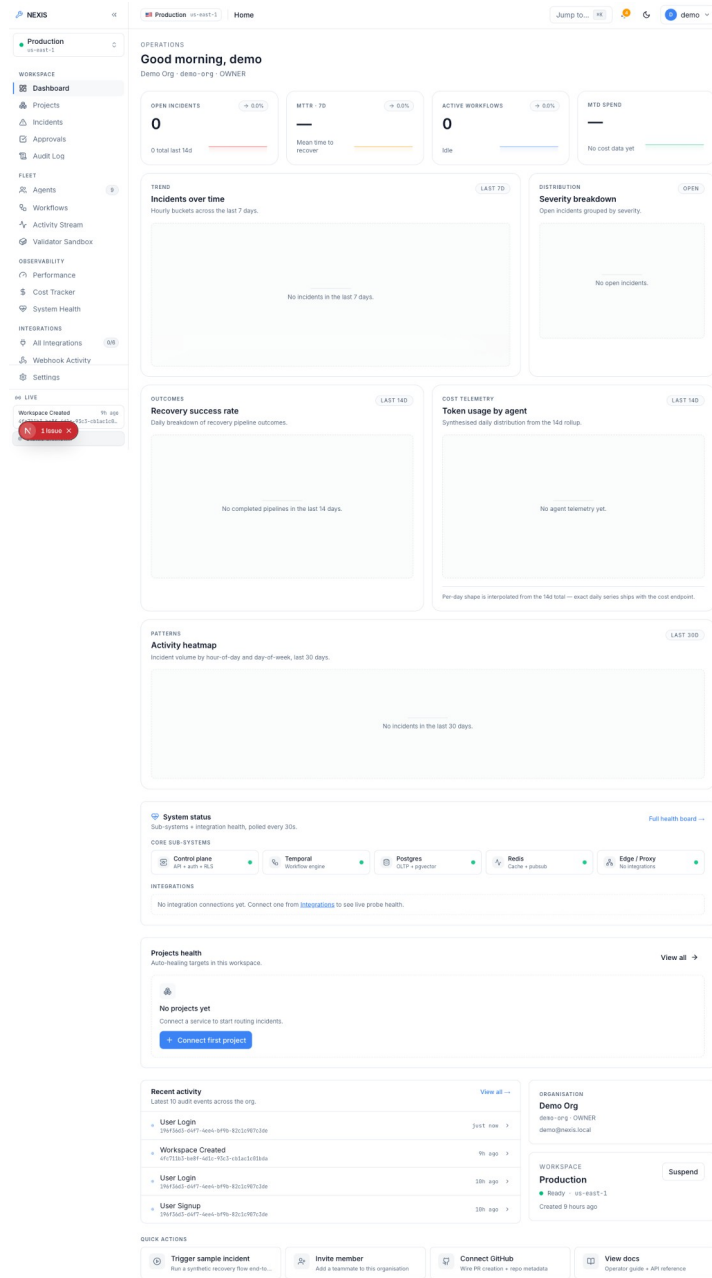


Figure 5.4: Administrator Dashboard. (KPI tiles, incidents-over-time chart, and system status panel).

Moving to the Agents page (Figure 5.5), the nine-agent fleet is listed across both layers: the five Execution Team agents (Architect, Backend, QA, DevOps, Data Engineer) and the Self-Healing Loop agents (Sentinel, Pathfinder, Synthesiser, Validator), each with its role and current status. The Workflows page (Figure 5.6) lists RecoveryPipeline executions, every ingested incident appears on the Incidents page (Figure 5.7), and the Activity Stream (Figure 5.8) carries the organisation-wide live event feed that both humans and agents write into.

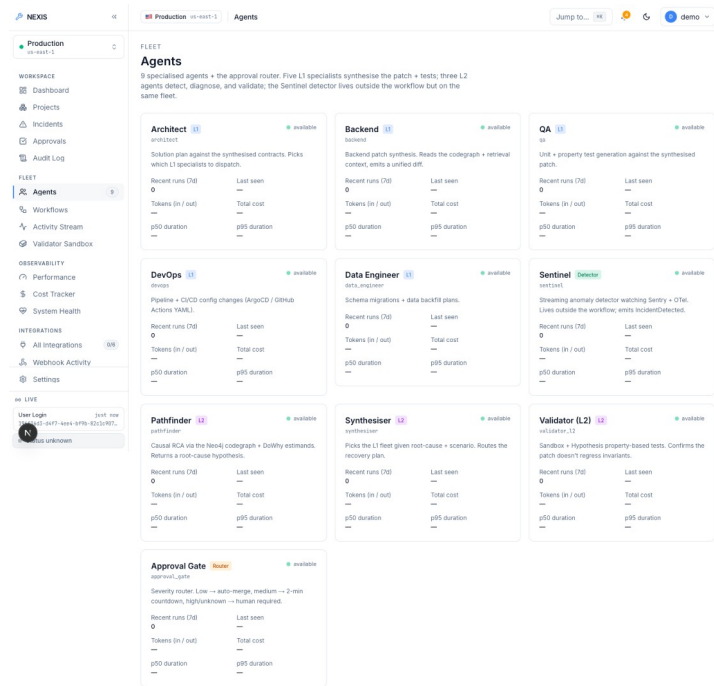


Figure 5.5: Agents Screen. (the nine-agent fleet roster across both layers).

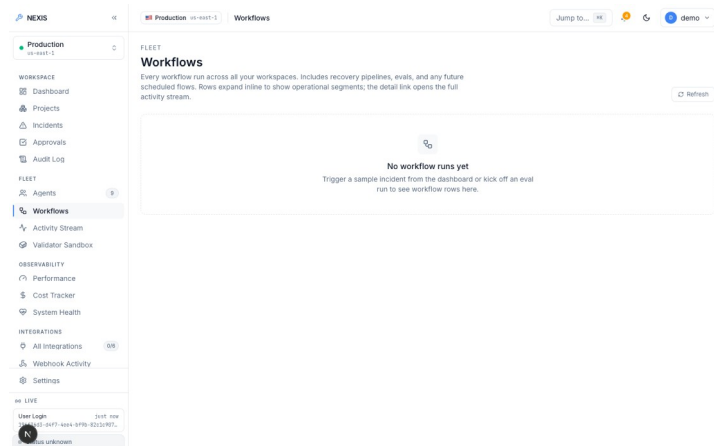


Figure 5.6: Workflows Screen. (RecoveryPipeline run listing).

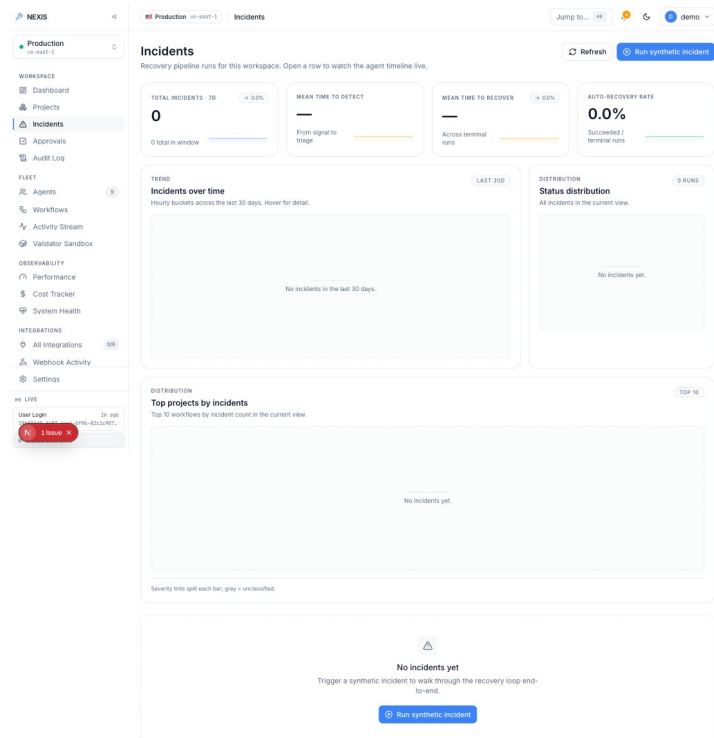


Figure 5.7: Incidents Screen. (the organisation-wide incident list).

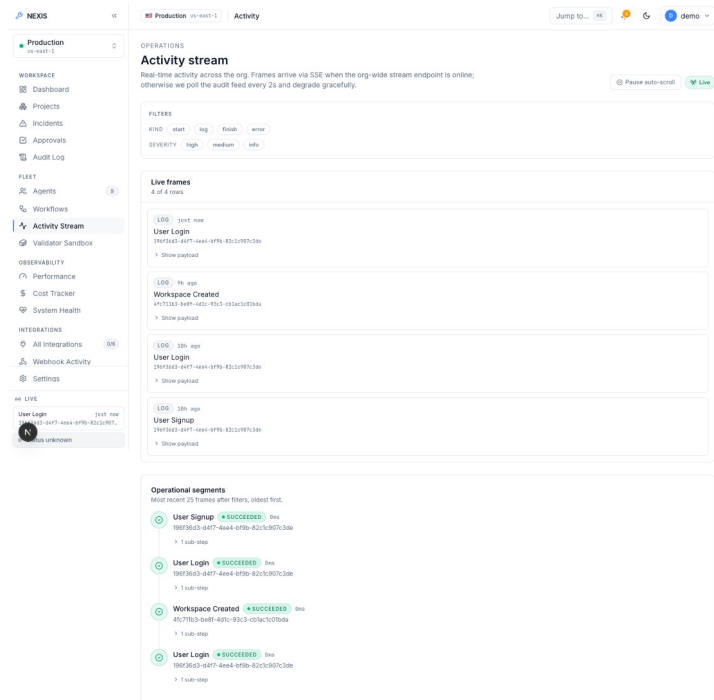


Figure 5.8: Activity Stream. (organisation-wide live event feed).

Membership is managed under Settings (Figure 5.9), where the owner / admin / member roles sit beside a genuine heuristic role-recommendation panel rather than a mock: three conditions trigger a recommendation — a non-administrator who has logged five or more administrator-gated audit actions in the past 30 days, an administrator dormant for 90 days or more, and an active administrator with no administrator-gated actions at all. Each

recommendation states the evidence behind it in plain language; accepting applies the role change under an optimistic-concurrency guard, and dismissing starts a 30-day per-rule cool-down. Sessions (Figure 5.10) are listed with device, IP address, and last-seen time and can be revoked remotely, with the caller’s own session badged “This device”; alongside them sit scoped API keys (`nx_live_...`) and Stripe billing [13].

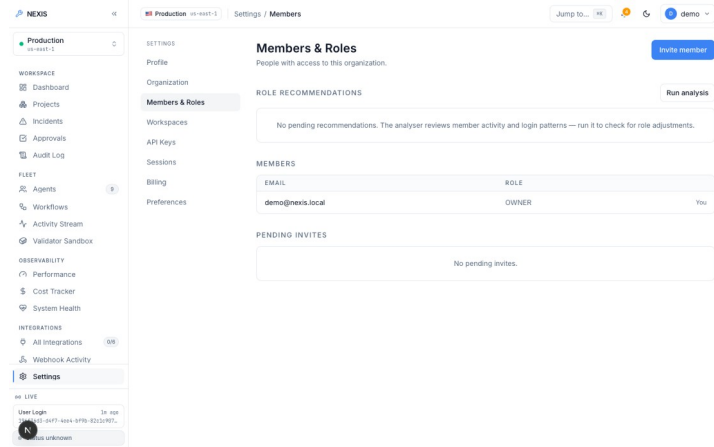


Figure 5.9: Members Settings. (members list with the heuristic role-recommendation panel).

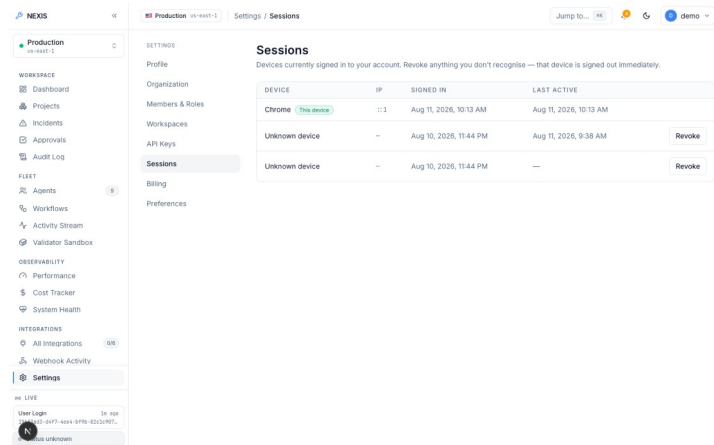


Figure 5.10: Sessions Settings. (active sessions with device, IP, and remote revoke).

5.2.3 Self-Healing Loop — Live Evidence

This subsection carries the evidentiary weight: two real, end-to-end executions of the nine-agent RecoveryPipeline, run against a live local LLM (Ollama, `qwen2.5-coder:14b` and `gpt-oss:20b`), are shown exactly as they occurred, one safely auto-rejected at the approval gate and the other approved and fully succeeded.

Both runs start from the Live Demo console (Figure 5.11), which offers 27 curated fault-scenario cards drawn from a fixture taxonomy spanning application, database, deploy, observability, and security categories. Of those 27, exactly one is wired end-to-end today; the rest are UI-complete but backend-pending, each labelled “Coming soon” in the interface itself. Both runs documented below were triggered through that one wired path, and Figure 5.12 shows the pipeline structure they execute.

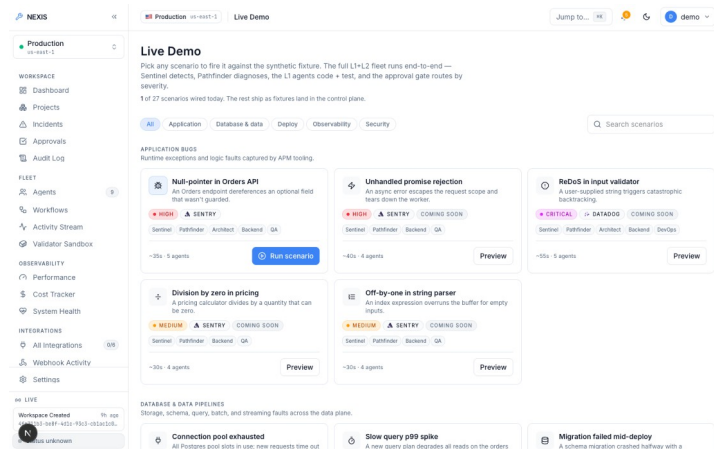


Figure 5.11: Live Demo Console. (27-scenario fault-injection picker; one scenario fully wired, the rest backend-pending).

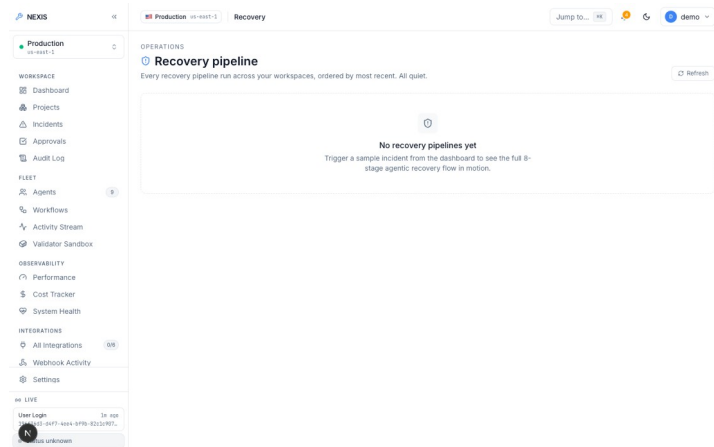


Figure 5.12: Recovery Pipeline Screen. (the nine-agent pipeline structure).

Figure 5.13 catches the first run genuinely mid-flight, with Architect, Backend, and QA already completed and showing real, non-zero token counts and costs from actual LLM calls, while DevOps and Data Engineer are still running in parallel. A stubbed pipeline would report zero tokens; consequently, these figures are themselves evidence. A little later in the same run (Figure 5.14), every Layer-1 and Layer-2 step has completed green — patch generated, property-tested, and sandbox-validated — leaving severity at “Medium” and approval at “Pending” inside the 120-second window.

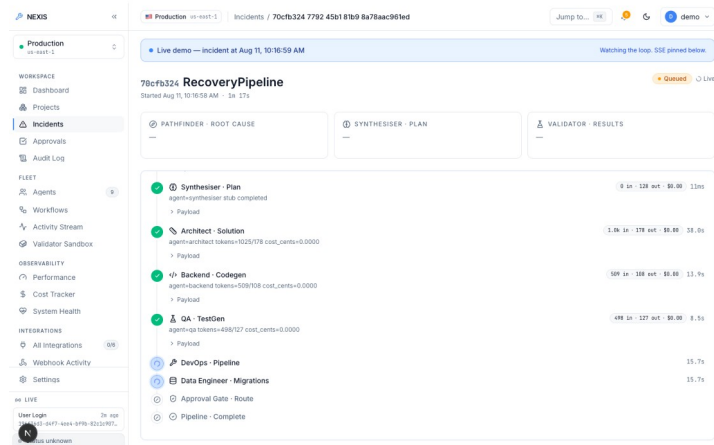


Figure 5.13: Incident Detail, Mid-Flight. (completed Architect/Backend/QA steps with real token counts; DevOps and Data Engineer running in parallel).

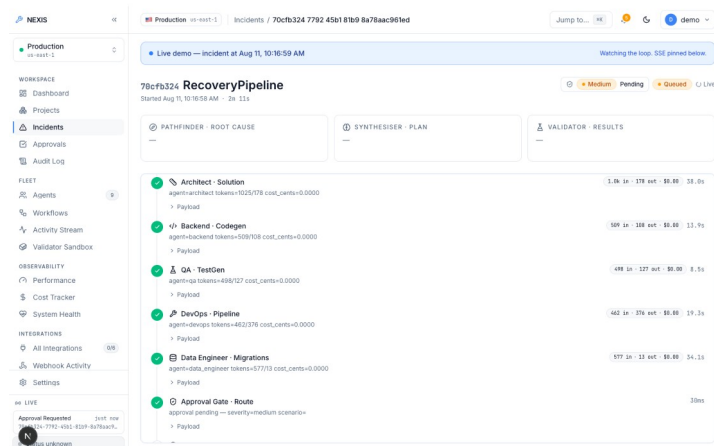


Figure 5.14: Incident Detail, Pipeline Complete. (all Layer-1 and Layer-2 steps green; severity Medium, approval Pending).

That window elapsed with no decision, and the workflow terminated as `timeout_rejected` (Figure 5.15) — an honest negative result, reported deliberately: absent a human decision, the system refuses to deploy unreviewed code rather than proceeding. A pipeline that silently auto-deployed on timeout would itself be the defect.

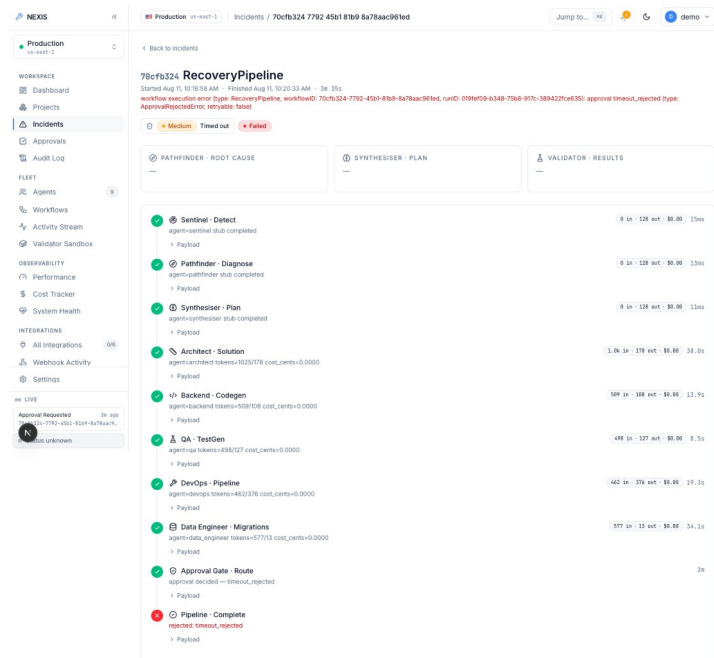


Figure 5.15: Timeout-Rejected Run. (the 120-second approval window elapsed; the workflow safely auto-rejected as `timeout_rejected` rather than deploying unreviewed code).

The second run supplies the positive counterpart to this result: approved within the window, it records “Medium” severity, “Approved”, “Succeeded”, and a full nine-step timeline completing in 1 minute 40 seconds, with real per-agent token and cost figures throughout (Figure 5.16). Whether the patch is genuinely model-written is settled by Figure 5.17, the expanded raw JSON of the Backend step: it holds the unified diff produced by `qwen2.5-coder:14b` with `provider="ollama"` recorded in the payload, parsed model output rather than a fixture template.

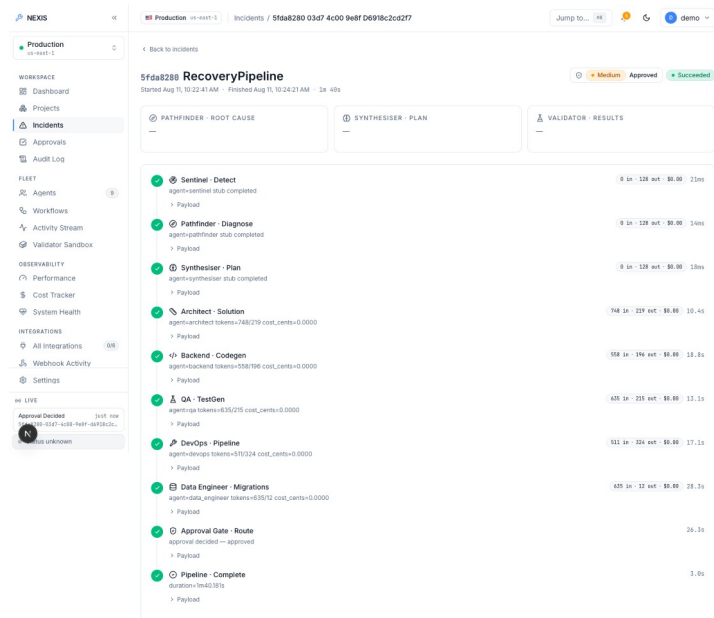


Figure 5.16: Fully Approved Run. (Medium severity, Approved, Succeeded; complete nine-step timeline in 1 minute 40 seconds).

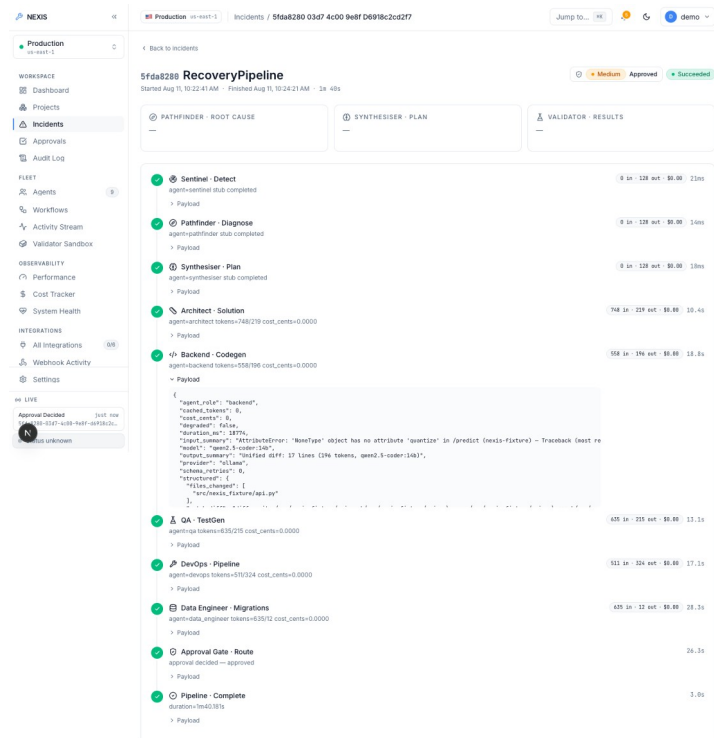


Figure 5.17: Backend Payload, Expanded. (raw JSON evidence of the qwen2.5-coder:14b-generated unified diff, provider=“ollama”).

Pending runs surface in the Approvals queue (Figure 5.18), where three human actions are available (Reject, Modify, Approve), each decided in the dialog shown in Figure 5.19. Modify is where the Reinforcement Learning from Human Feedback (RLHF) mechanism becomes concrete: the engineer edits the LLM-generated unified diff inside the embedded editor before approving it (Figure 5.20), and that edited diff, along with every terminal decision, is persisted to `feedback_examples`, feeding a JSONL export for future fine-tuning. Submitting a paid fine-tuning job, however, remains a deliberate manual step rather than an automated one.

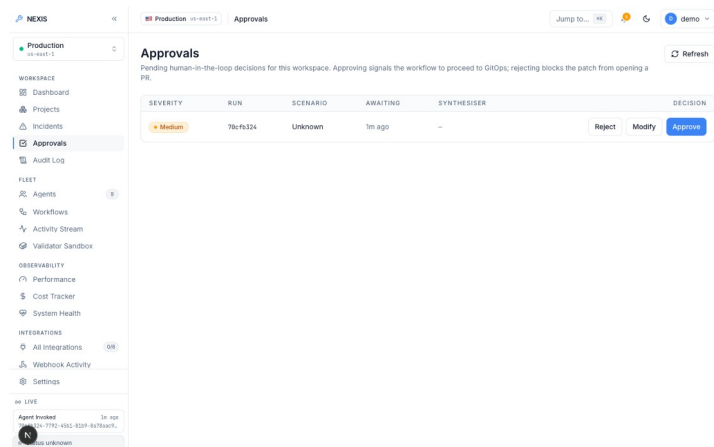


Figure 5.18: Approvals Queue. (real pending approval row with Reject / Modify / Approve actions).

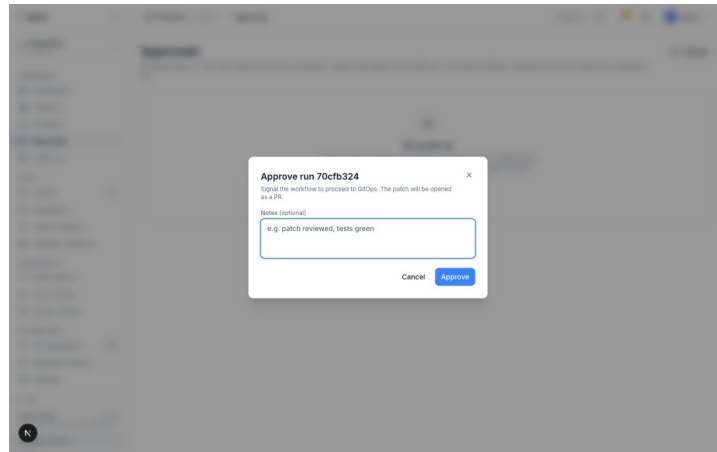


Figure 5.19: Approval Decision Dialog. (the Reject / Modify / Approve decision surface for a pending patch).

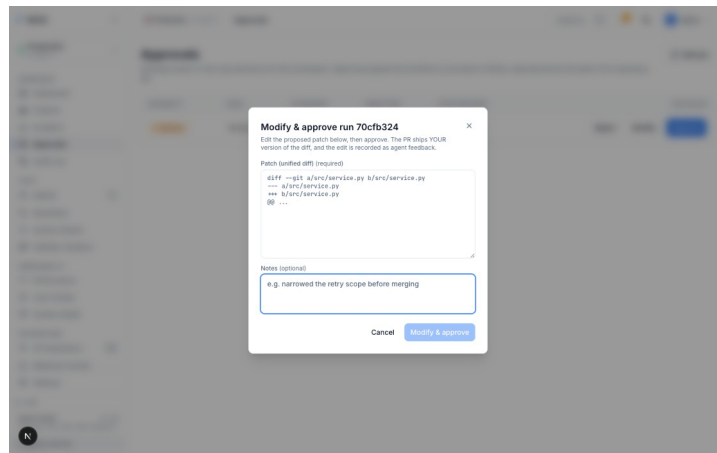


Figure 5.20: Modify-and-Approve Dialog. (RLHF diff editing; the engineer’s edit becomes a feedback_examples training row).

Nothing reaches that queue unvalidated. The Validator sandbox console (Figure 5.21) fronts the shadow-execution service in which every candidate patch is run against its QA-generated property tests, inside a container with no network, a read-only filesystem, and all capabilities dropped. Each agent, in turn, has its own detail page; Figure 5.22 shows the Backend agent’s configuration and run history.

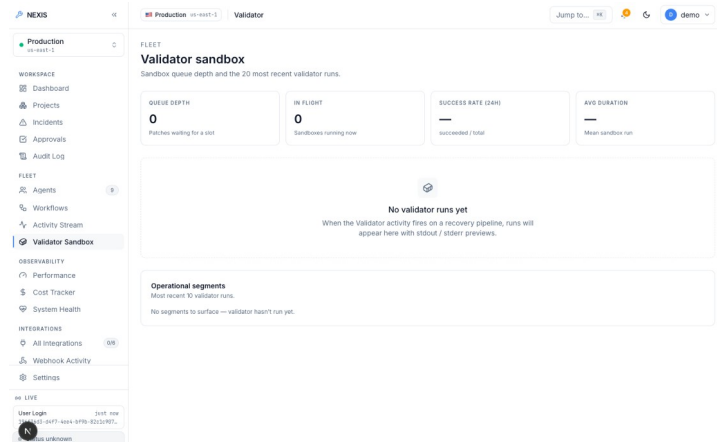


Figure 5.21: Validator Sandbox Console. (sandboxed shadow execution of candidate patches).

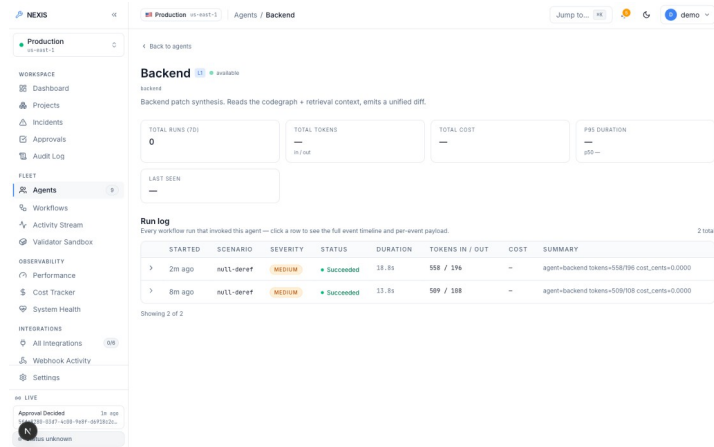


Figure 5.22: Agent Detail, Backend Agent. (per-agent configuration and run history).

5.2.4 Docker Preview-Deploy Engine (Ops Tab)

Deploy action, live status, preview URL, logs, and history all live on the project’s Ops tab, which fronts the preview-deploy engine; Figure 5.23 shows it in its empty state, before any deployment exists. Attempting Deploy on a project with no GitHub App connection (Figure 5.24) returned the real validation error “project has no github repository bound”, surfaced verbatim. This is *not* a successful deploy; rather, it is the validation chain correctly rejecting a request it cannot fulfil, and the limitation is environmental (no GitHub App credentials in the evaluation environment) rather than a defect in the engine.

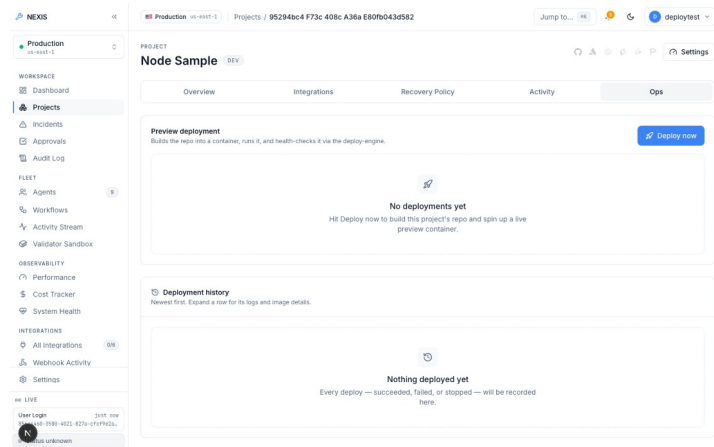


Figure 5.23: Project Ops Tab, Empty State. (the Deploy panel before any deployment).

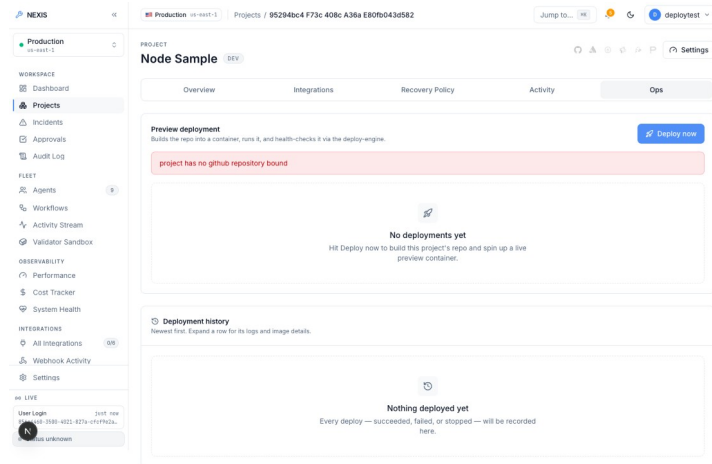


Figure 5.24: Project Ops Tab, Deploy Attempt. (a real validation error — “project has no github repository bound” — returned by the live API).

Independently of that limitation, the engine was proven live (Listing 5.1): `heroku/node-js-getting-started`, which ships without a Dockerfile, was cloned, detected as Node.js, handed a generated Dockerfile, built, and run under resource limits, and the preview URL served the app’s real rendered HTML before a clean stop and removal.

```
POST /v1/deploy (repo: heroku/node-js-getting-started, no Dockerfile present)
-> 200 OK
{"status": "running", "dockerfile_source": "generated", "detected_stack": "node",
"url": "http://localhost:54671/", "image_tag": "nexus-preview-test-project:22d06076c357"
build_log: "...npm ci... EXPOSE 3000... CMD [\"npm\", \"start\"]..."
container_log: "Listening on 3000\nRendering 'pages/index' for route '/'"}

GET http://localhost:54671/ -> real rendered HTML of the Node app (verified)

POST /v1/deploy/test-e2e-001/stop -> {"status": "stopped"}; container removed.
```

Listing 5.1: Verified live deploy-engine run.

Per-project self-healing behaviour is configured on the Recovery Policy tab (Figure 5.25), which holds the auto-rollback-on-SLO-breach switch and the kill switch that suspends autonomous recovery entirely. A failed build or a failed health check, meanwhile, inserts a real incident with source: `"deploy_engine"` into the very same Sentinel-to-RecoveryPipeline path used by every other incident source, so a broken build becomes an ordinary self-healing target.

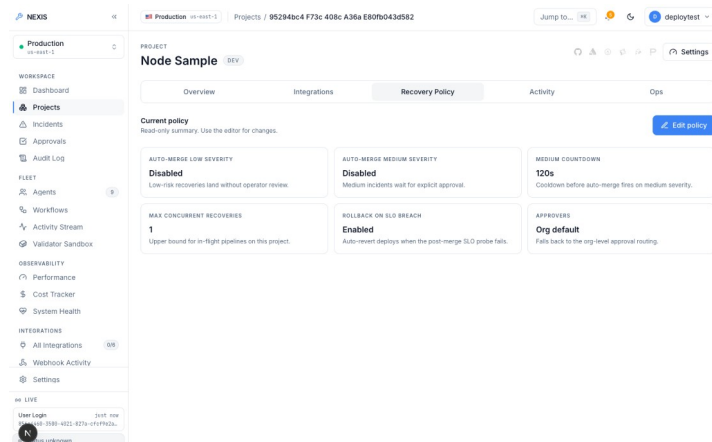


Figure 5.25: Project Recovery Policy Tab. (per-project self-healing configuration).

5.2.5 Integrations and Observability

External providers are managed from the Integrations page (Figure 5.26): the GitHub App (installation, repository listing, webhook-driven incidents on merged pull requests); Sentry [14], Datadog [16], and PagerDuty [15], each a webhook incident source; Slack [17], with OAuth install and one-click approve/reject from a Slack message; Argo CD; and Stripe. Companion pages log webhook deliveries and probe integration health.

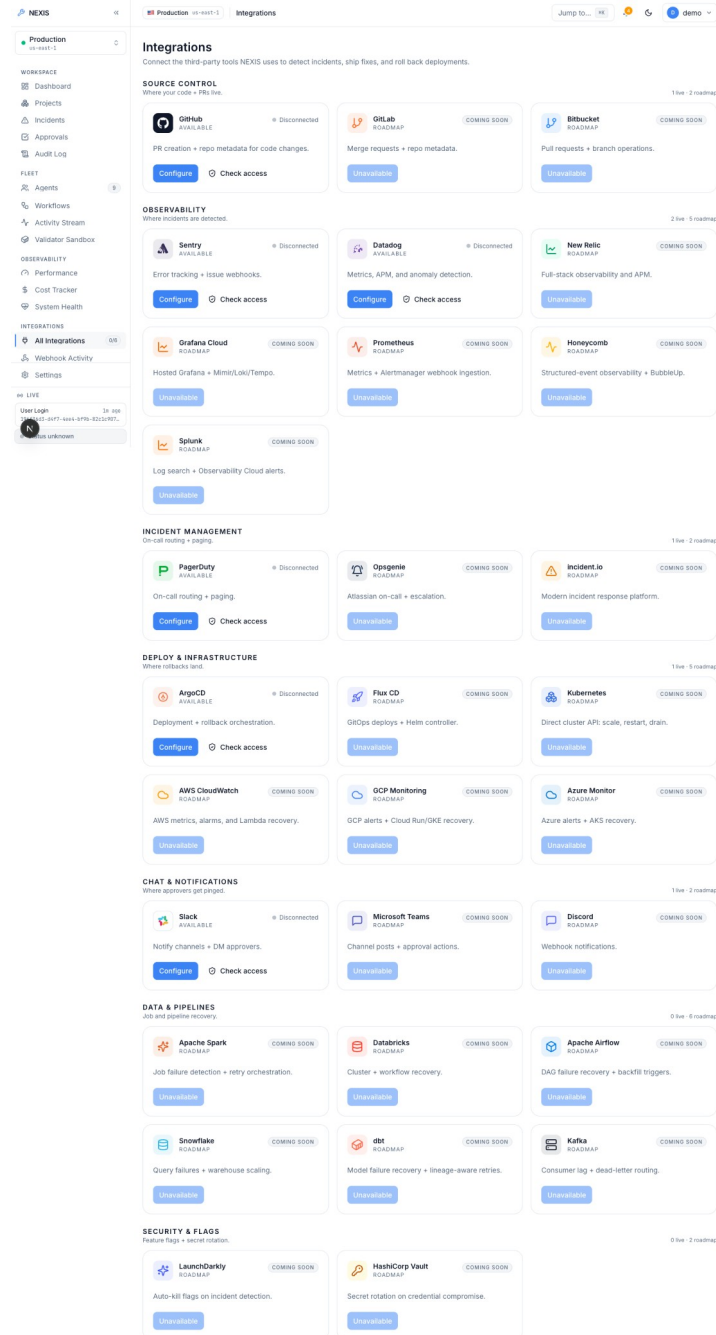


Figure 5.26: Integrations Screen. (provider catalogue: GitHub App, Sentry, Datadog, PagerDuty, Slack, Argo CD, Stripe).

On the observability side sit performance charts, a per-organisation token cost tracker with a daily budget, and the System Health page (Figure 5.27), which fans out parallel dependency probes against PostgreSQL, Redis, Neo4j, MinIO, and Temporal under a bounded timeout and reports each result independently. The Eval Harness (Figure 5.28) runs side-by-side OpenAI-versus-Ollama comparisons across fixture incidents, persisting transcripts, tracking cost, and offering administrator-only CSV export, while a pgvector-backed knowledge base [19] serves retrieval for the agents.

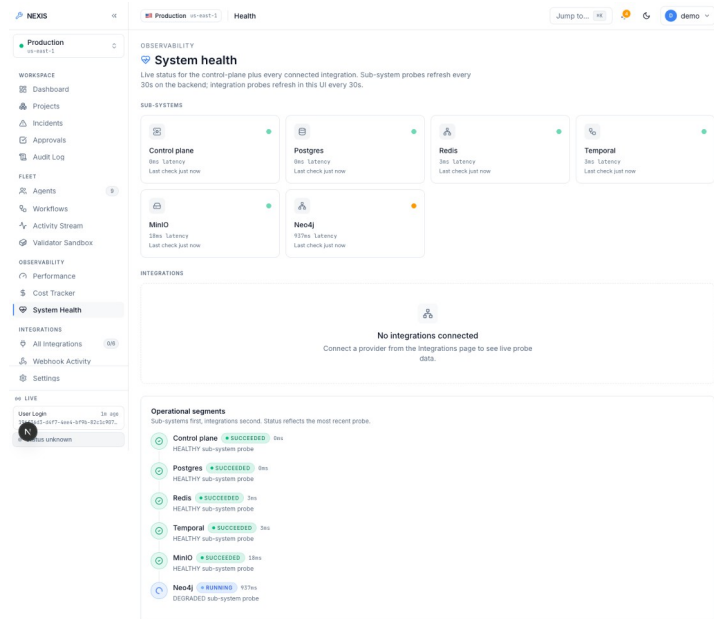


Figure 5.27: System Health. (parallel dependency probes across PostgreSQL, Redis, Neo4j, MinIO, and Temporal).

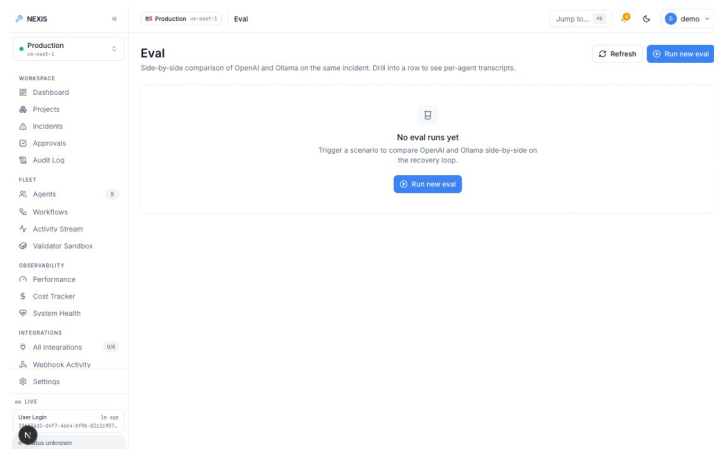


Figure 5.28: Eval Harness. (side-by-side OpenAI-versus-Ollama comparison over fixture incidents).

Append-only and hash-chained, the Audit Log (Figure 5.29) builds each entry on its predecessor's hash; thus, tampering breaks the chain and becomes detectable through the verify endpoint. CSV export is also available for offline review.

Timestamp	Actor	Action	Target
Aug 11, 2026, 10:13:20 AM	19e736d3...	user.login	19e736d3-6d77-4aea-979b-82c1c907c1de
Aug 11, 2026, 01:00:04 AM	19e736d3...	workspace.created	4ff71285-9d8f-4832-93c3-c51ac1d316b9
Aug 10, 2026, 11:44:52 PM	19e736d3...	user.login	19e736d3-6d77-4aea-979b-82c1c907c1de
Aug 10, 2026, 11:44:30 PM	19e736d3...	user.signup	19e736d3-6d77-4aea-979b-82c1c907c1de

Figure 5.29: Audit Log. (append-only, hash-chained entries with CSV export).

5.3 Database Schema

State lives in PostgreSQL 17, spread across 33 sequential migrations. Multi-tenancy is enforced at the database level itself: nearly every table carries an organisation identifier, RLS policies [7] restrict queries to the calling organisation’s rows, and a request-scoped transaction binds `app.current_org_id`. As such, even a query omitting an organisation `WHERE` clause cannot touch another tenant’s data. Two exceptions are deliberate: `users` remains a global identity, with tenancy applied through `org_members`, while `sessions` stays user-scoped — a login session belongs to a person, not a tenant. The most important tenant-scoped tables are set out in Table 5.1.

Table 5.1: Key tenant-scoped tables in the Nexis PostgreSQL schema.

Table	Purpose	RLS scope
<code>incidents_raw</code>	Every ingested fault event, from webhooks, fixtures, and the deploy engine	Organisation-scoped
<code>workflow_runs</code>	One row per Temporal RecoveryPipeline execution, with per-step activity events	Organisation-scoped
<code>approval_decisions</code>	Terminal approval-gate outcomes: approve, reject, modify, timeout	Organisation-scoped
<code>feedback_examples</code>	RLHF training rows persisted from every terminal approval decision	Organisation-scoped
<code>audit_log</code>	Append-only, hash-chained record of privileged actions	Organisation-scoped

Neo4j holds the dependency graph that the Pathfinder traverses, MinIO the encrypted patch blobs referenced from `workflow_runs`, and pgvector columns back the knowledge-base retrieval; Temporal’s durable event history, moreover, can reconstruct the per-step records in `workflow_runs` in a recovery scenario.

Chapter 6: Testing and Results

This chapter sets out the verification evidence as it was gathered: static checks across every service, live end-to-end runs driven by a real local Large Language Model (LLM) rather than stubs or mocks, and the defects that verification turned up.

6.1 Static Verification

Each component was verified through its own native toolchain rather than one uniform check. Across the four Go modules, `go build ./...`, `go vet ./...`, and `go test ./...` were run in sequence, a pass requiring zero build errors, zero vet findings, and zero failing tests; the two Python components were verified with `pytest`; and the Next.js console was put through its three gate commands, TypeScript type checking, ESLint, and a full production build. Table 6.1 sets out each command against the component it was run on, together with the outcome.

Table 6.1: Static verification commands and results.

Component	Command	Result
services/control-plane — Go	<code>go build ./... && go vet ./... && go test ./...</code>	Clean — build, vet, and all tests pass
services/validator — Go	<code>go build ./... && go vet ./... && go test ./...</code>	Clean — build, vet, and all tests pass
services/gitops — Go	<code>go build ./... && go vet ./... && go test ./...</code>	Clean — build, vet, and all tests pass
services/deploy-engine — Go	<code>go build ./... && go vet ./... && go test ./...</code>	Clean — build, vet, and all tests pass
services/causal-inference — Python	<code>pytest</code>	38 tests, all passing
Validator Hypothesis sidecar [9] — Python	<code>pytest</code>	All tests passing
apps/web — Next.js	<code>pnpm typecheck && pnpm lint && pnpm build</code>	Clean — type check, lint, and production build all pass

The result column is not aspirational: every command listed was run against the final codebase and returned clean, the Go suites including the JSON Web Token (JWT) clock-injection tests discussed in Section 6.4, which were silently failing earlier in the cycle and genuinely pass now. Static verification reaches every deployable unit of the platform, so no service ships without first clearing its own build, its own static analysis, and its own test suite.

6.2 Live End-to-End Recovery Runs

Static verification establishes per-service soundness, not that the nine agents, the Temporal workflow [2], the validator sandbox, the approval gate, and the live console actually cooperate. Two recovery runs were therefore carried out against the real system: genuine PostgreSQL with Row-Level Security switched on, genuine Temporal workflows, a genuine Neo4j graph, and a real local LLM (Ollama [20] serving `qwen2.5-coder:14b` and `gpt-oss:20b`) generating the actual plans, diffs, and tests. Nothing was mocked or replayed.

6.2.1 Run 1: Approval-Gate Timeout — the Safety Mechanism Under Real Timing Pressure

A fault fixture was injected, Sentinel detected it, and the pipeline ran to completion: the Architect produced a plan, the Backend agent a unified-diff patch, the Quality Assurance (QA) agent Hypothesis property tests, and the Validator executed the patch against them inside its locked-down container sandbox, with every Layer 1 and 2 step

turning up green. At the human approval gate, however, no decision arrived within the 120-second window, so the workflow resolved to a rejected terminal state.

This is a *positive* result: the central safety invariant is that no patch deploys without validation and approval, and that the system fails closed on any ambiguity, where an absent decision counts as refusal, never as consent. Rather than ship an unreviewed change, the system discarded a genuinely generated patch and durably recorded the timeout-rejected outcome. It is reported deliberately: a rejection is the stronger evidence that the gate is real.

6.2.2 Run 2: Approved and Succeeded in 1 Minute 40 Seconds

In the second run, the same detection path was followed, but approval was granted within the window through the console's decision dialog. Classified as Medium severity, the patch passed sandbox validation, and the pipeline proceeded through deploy to a *Succeeded* state, 1 minute 40 seconds end-to-end. Two properties set this run apart from a staged demonstration: the timeline records genuine per-agent token and cost figures from the live Ollama provider, and the raw JSON of the Backend agent's activity event contains the actual unified diff written by `qwen2.5-coder:14b (provider="ollama"` appears in its metadata): the deployed patch is the very patch the model wrote, not a canned fixture.

Between them, the two runs cover both branches of the approval gate — refusal by timeout on one side, explicit approval on the other — against one and the same real pipeline. These are exactly the two behaviours a closed-loop recovery system has to get right, and Chapter 5 reproduces the console evidence for each of them, from the mid-flight timeline with live per-agent token counts through to the two terminal states.

6.3 Live Deploy-Engine Verification

The deploy-engine was verified in its own live run against the public repository `heroku/node-js-getting-started`, chosen deliberately because it contains no `Dockerfile`, which forces the engine's hardest path: clone the repository, infer the stack from marker files, generate a working `Dockerfile` from nothing, build the image, run the container under resource limits, and only then prove liveness over HTTP. Listing 6.1 reproduces the request/response transcript of that run verbatim.

Listing 6.1: Live deploy-engine end-to-end run: clone, generate Dockerfile, build, run, HTTP-verify, stop.

```
POST /v1/deploy (repo: heroku/node-js-getting-started, no Dockerfile present)
-> 200 OK
  "status": "running", "dockerfile_source": "generated", "detected_stack": "node",
  "url": "http://localhost:54671", "image_tag": "nexis-preview-test-project:22d06076c357"
  build_log: "...npm ci... EXPOSE 3000... CMD [\"npm\\\", \"start\\\"]..."
  container_log: "Listening on 3000\nRendering 'pages/index' for route '/'"

GET http://localhost:54671/ -> real rendered HTML of the Node app (verified)

POST /v1/deploy/test-e2e-001/stop -> {"status":"stopped"}; container removed.
```

Every stage in that transcript genuinely took place: the stack was detected as Node.js, the generated Dockerfile's `npm ci`, `EXPOSE 3000`, and `CMD` directives appear in the build log actually executing, the container log shows the application booting and serving its index route, an independent HTTP GET on the preview URL returned the app's real rendered HTML, and the stop endpoint removed the container cleanly. The repository is public and the whole sequence is three HTTP calls, so this is a reproducible integration test rather than a one-off observation. One environmental substitution is worth flagging: Docker was unavailable on the development machine, so Podman [11] served as the runtime through the same Docker-compatible API [10] that both the deploy-engine and the Validator target, and the code path does not differ between them. The clone was unauthenticated, so the GitHub App installation-token path is present in the code but went unexercised, a point noted among the limitations.

6.4 Defects Found and Fixed During This Engineering Cycle

Verification surfaced two genuine defects (Table 6.2). The JWT defect was *silent*, failing without blocking anything visible, and it was caught only because the full `go test ./...` output was read rather than skimmed. The migration defect, by contrast, manifests only at apply time.

Table 6.2: Defects discovered by verification.

Defect	Root Cause	Fix	Outcome
13 JWT tests silently failing	exp claims checked against the wall clock, not an injected test clock	Inject clock via <code>jwt.WithTimeFunc</code> (one line)	All 13 pass deterministically; production behaviour unchanged
Migration failed to apply	Reserved PostgreSQL keyword <code>current_role</code> used unquoted as a column name	Column renamed to <code>existing_role</code>	Applies cleanly; Go suites remain green

One further bug, pre-existing and out of scope, was deliberately left unfixed: the workspace-onboarding poller receives a 404 from `/v1/workspaces/{id}/events`, though the workspace provisions server-side. This is recorded among the limitations.

6.5 Testing Summary

Static verification came back 100% clean across every service, and on top of it sit three live runs: two nine-agent recovery executions exercising both approval-gate branches, plus one deploy-engine run, none simulated, every LLM step served by a real local model. Equally important is what was *not* tested: no formal load or performance testing was carried out, no throughput or latency under multi-tenant traffic is claimed, and the 1-minute-40-second figure reflects a single observed run, not a statistical claim. Load testing and a larger sample of recovery runs, sufficient to support meaningful Mean Time To Recovery (MTTR) distributions, remain future work (Chapter 7).

Chapter 7: Conclusion and Future Work

This chapter closes the report by measuring the delivered system against the objectives set out in Section 1.4, and by drawing the boundary of the evidence exactly where the evidence stops. Section 7.1 reviews what was built and demonstrated, Section 7.2 states every known limitation in plain terms rather than softened ones, Section 7.3 turns each of those limitations into a concrete next step grounded in the existing architecture, and Section 7.4 states the project’s central claim, deliberately narrow and deliberately defensible.

7.1 Overview

This project set out to close a loop that existing Artificial Intelligence (AI) developer tools leave open: from a detected software failure to a deployed, verified fix, with a human retained as a single decision point rather than the pipeline itself. Nexis, the platform delivered here, realises that goal as a working system, not a design proposal.

The central deliverable is the nine-agent closed-loop self-healing pipeline, durably orchestrated by Temporal [2]. A five-agent Execution Team (Architect, Backend, QA, DevOps, Data Engineer) carries out the engineering work, while the four-agent Self-Healing Loop (Sentinel, Pathfinder, Synthesiser, Validator) detects faults, localises root causes over the Neo4j dependency graph [3], sandbox-validates every candidate patch, and routes it through a severity-classified Approval Gate offering Approve, Reject, and Modify. Chapter 6 exercised this pipeline live in two end-to-end recovery runs against a real local language model [20]. In one run, the patch was timeout-rejected exactly as the safety design intends, its 120-second approval window having elapsed; in the other, a detected fault travelled through planning, patch generation, property-based test validation, and human approval to a Succeeded deployment in roughly one minute and forty seconds.

The second major deliverable is a from-scratch, Docker-compatible preview-deploy engine [10]: it detects a repository’s stack, generates a missing Dockerfile, builds and serves the container under resource limits, and returns a live preview URL. Its build and health-check failures feed back into the same self-healing loop, and the engine was proven live against a public repository that had no Dockerfile. Around both sits the complete multi-tenant platform: SSO/MFA authentication [12], row-level RBAC [7], Stripe billing [13], integrations with GitHub, Sentry, Datadog, PagerDuty, and Slack [14, 16, 15, 17], and OpenTelemetry observability [5] backed by a hash-chained audit log, all of it passing full build, lint, and test suites.

7.2 Limitations of the Project

Academic integrity demands that the boundary between what was demonstrated and what was not be stated in plain terms, not softened by hedging. As such, every limitation below is real, known, and deliberate; none of it is dressed up in gentler wording.

- **No production credentials were exercised.** The evaluation environment used for this report held no real GitHub App or OpenAI production credentials. Live evidence was produced instead with local Ollama models [20] and an unauthenticated clone of a public repository, so a fully credentialed production path, ending in a real, merged GitHub pull request, was never exercised end-to-end.
- **The platform is not yet live on its domain.** The `nexis.sh` domain is registered, but the platform has not yet been deployed to it, and every piece of evidence in this report comes from local, native execution. Docker itself was unavailable in the build environment, so Podman [11] was substituted as a runtime compatible with the Docker Application Programming Interface (API) and shown to behave identically.
- **Fine-tuning is a manual step by design.** The Reinforcement Learning from Human Feedback (RLHF) data-export pipeline is real and has been tested: every terminal approval decision, including engineer-edited diffs, is persisted and exportable as JSON Lines (JSONL). Submitting a fine-tuning job against that export, however, is left as a deliberate manual step, since it involves real money and a real external API call rather than an automated one.

- **Only one of twenty-seven demo scenarios is fully wired.** The Live-Demo console lists twenty-seven curated fault scenarios, of which exactly one is wired end-to-end to a real fixture and classification path today. The remaining twenty-six are complete on the User Interface (UI) side but still backend-pending.
- **The causal ranking is a proxy, not a fitted causal model.** The causal-inference sidebar ranks root-cause candidates through an honest statistical and graph-structural scoring method, backed by a transparent, per-component confidence breakdown. Explicitly, it is not a fitted DoWhy structural causal model [4], since no interventional production data exists against which such a model could be fitted, and claiming otherwise would overstate what has been built.
- **A known, unrelated bug was left unfixed.** A pre-existing bug in the workspace-onboarding progress poller, a 404 returned from `/v1/workspaces/{id}/events`, turned up during evaluation and was intentionally left unfixed as out of scope. Regardless, the workspace itself provisions successfully on the server side.
- **No formal load testing was performed.** As Chapter 6 notes, no formal concurrent-load performance benchmark was run, and the latencies reported there come from single-run live executions rather than from a controlled load harness.

7.3 Future Work

Each item below is grounded in the existing architecture: it names a concrete next step rather than a generic aspiration.

- **Credentialed production deployment** to the registered `nexis.sh` domain, with a real GitHub App and a paid OpenAI account, is the only way to exercise authenticated cloning, real pull-request creation, and merge-triggered redeploy end-to-end.
- **Complete the scenario library.** Wiring the remaining twenty-six Live-Demo cards to real fixtures would extend fault-injection coverage across all five scenario categories.
- **Per-service Sentinel baselines.** Splitting the per-organisation EWMA baseline down to the level of each service would stop one noisy service from masking a quiet one.
- **A fitted causal model.** Once interventional production data exists, a real DoWhy model [4] can be fitted and evaluated against the deliberately replaceable graph-evidence proxy.
- **Execute a fine-tuning run** on the RLHF export to measure whether a tuned model lowers the rejection and modification rates seen at the gate.
- **Real preview subdomains**, served through a Traefik-style reverse proxy instead of `localhost` ports, would let previews be shared beyond one machine.
- **Formal load benchmarking** to characterise throughput, queueing behaviour under simultaneous incidents, and Temporal worker saturation.
- **Human-subjects usability study.** The NASA-TLX instrument [18] is already wired into the post-incident flow; only recruitment and analysis remain.

7.4 Conclusion

The central claim is deliberately narrow, yet deliberately strong. A software fault can travel from automatic detection, through multi-agent diagnosis, patch generation, and sandboxed validation, past a single human approval, to a deployed fix, inside one continuous, durable pipeline. This is demonstrated rather than asserted: once, the loop refused to proceed when its approval gate went unanswered; once, it carried a real fault to an approved, deployed resolution in under two minutes. Equally deliberate is what the project does not claim. Section 7.2 draws the evidence boundary exactly where the evidence stops, and within it, Nexis thus shows that human-gated, closed-loop fault recovery is buildable today.

References

1. GitHub Copilot. AI pair-programming assistant for code completion and suggestion. Available at: <https://github.com/features/copilot> (Accessed: 11 August, 2026).
2. Temporal. Durable workflow orchestration engine for fault-tolerant distributed applications. Available at: <https://temporal.io/> (Accessed: 11 August, 2026).
3. Neo4j. Native graph database for storing and querying highly connected data. Available at: <https://neo4j.com/> (Accessed: 11 August, 2026).
4. DoWhy. Causal inference library from Microsoft Research / PyWhy for modelling and estimating causal effects. Available at: <https://github.com/py-why/dowhy> (Accessed: 11 August, 2026).
5. OpenTelemetry. Open-source observability framework for traces, metrics, and logs. Available at: <https://opentelemetry.io/> (Accessed: 11 August, 2026).
6. Argo CD. Declarative GitOps continuous delivery tool for Kubernetes. Available at: <https://argo-cd.readthedocs.io/> (Accessed: 11 August, 2026).
7. PostgreSQL Row Security Policies. Official documentation on row-level security for multi-tenant data isolation. Available at: <https://www.postgresql.org/docs/current/ddl-rowsecurity.html> (Accessed: 11 August, 2026).
8. OpenLineage. Open standard for data lineage collection and analysis. Available at: <https://openlineage.io/> (Accessed: 11 August, 2026).
9. Hypothesis. Property-based testing library for Python. Available at: <https://hypothesis.readthedocs.io/> (Accessed: 11 August, 2026).
10. Docker. Container platform for building, shipping, and running applications. Available at: <https://www.docker.com/> (Accessed: 11 August, 2026).
11. Podman. Daemonless, Docker-API-compatible container engine. Available at: <https://podman.io/> (Accessed: 11 August, 2026).
12. WorkOS. Enterprise-ready authentication and single sign-on platform. Available at: <https://workos.com/> (Accessed: 11 August, 2026).
13. Stripe. Online payment processing and billing infrastructure. Available at: <https://stripe.com/> (Accessed: 11 August, 2026).
14. Sentry. Application error tracking and performance monitoring platform. Available at: <https://sentry.io/> (Accessed: 11 August, 2026).
15. PagerDuty. Incident response and on-call management platform. Available at: <https://www.pagerduty.com/> (Accessed: 11 August, 2026).
16. Datadog. Cloud-scale monitoring and observability service. Available at: <https://www.datadoghq.com/> (Accessed: 11 August, 2026).
17. Slack API. Platform APIs for building Slack integrations and interactive messages. Available at: <https://api.slack.com/> (Accessed: 11 August, 2026).
18. NASA Task Load Index (NASA-TLX). Subjective workload assessment instrument developed by NASA Ames Research Center. Available at: <https://humansystems.arc.nasa.gov/groups/tlx/> (Accessed: 11 August, 2026).
19. pgvector. Open-source vector similarity search extension for PostgreSQL. Available at: <https://github.com/pgvector/pgvector> (Accessed: 11 August, 2026).

20. Ollama. Tool for running large language models locally. Available at: <https://ollama.com/> (Accessed: 11 August, 2026).